FINAL REPORT HDIP 24 JOE POWER 20109022 PROJECT ANDROIDS

(Aeronautical Navigation Data for Radio Officer Information Display System)

TABLE OF CONTENTS

Table of Contents

Landing Page: HDIP 24 Project Page

Landing Page: [HDIP 24 Project Page]

TABLE OF FIGURES

BACKGROUND

The purpose of this project is the development of a new information display screen (IDS) for an Aeronautical Radio Communication facility (Airnav Ireland, 2025). The system will display essential data for Aeronautical Radio Officers related to HF (High Frequency) and VHF (Very High Frequency) Radio, Weather Info, NOTAM (Notice to Airmen) and other operational and Engineering Notes. It is used to pass accurate and timely information to aircraft transiting the North Atlantic over radio.

INTRODUCTION

The project is based on the course work and an existing system. It was built using the Full Stack I course as a guideline. It is tailored to fit requirements of the organisation and aviation regulations.
A Model-View-Controller structure was implemented to organise routes, views, and data models for key components such as HF Stations and ATC Centers. This allowed the application to reach a usable dashboard early in development, providing a foundation for further expansion, testing, and integration.
The backend architecture, built with Hapi.js, supports secure input handling, validation, and role-based access—laying the groundwork for the later Svelte and map-based frontend.
This project has a two-phase development process:
Phase I establishes a secure, validated, fully functional backend.
Phase II builds a modern, interactive, map-driven interface powered by Svelte and Leaflet.

PROJECT PLANNING

IDS Phase I HAPI

The first concept is to have an input based hapi application which will render in html on a page. This is the backend and the basic requirement of the project. The plan is to have a Screen to render operational HF (High Frequency) And VHF (Very High Frequency) Radio Frequencies. This system currently exists but is near end of life and needs updating.

Key Deliverables

User Authentication. User Creation and Session Management. Password Hashing and Salting. Navigation bar with page-coloured tabs. Modular routing.
Input forms created and rendered for HF Stations, ATC (Air Traffic Control) Centers, Nat (North Atlantic) Tracks and Miscellaneous. Update and delete to complete CRUD (Create, Read, Update, Delete) requirements. Data Seeding.
Testing added: Model and API (Application Programme Interface). Joi Schemas with validation.
Child MVC (Model View Controller) structure to add additional notes to Stations and Centers
Error handling
Role Based Access Control
API construction

IDS Phase 2 Svelte

Cesium or Leaflet Map
Fetch API from local input and ADSB (Automatic Dependent Surveillance Broadcast) from open source
Add map features and markers and distance calculations

Landing Page: HDIP 24 Project Page

Research and Analysis

The current Information Display System is closed system. It relies on manual input at an admin pc and document upload via a thumb stick. It is envisaged that this will at least have network access for file upload.
The upgraded system could be stand alone or mounted on Azure / SharePoint which is extensively used with the company. Future integration with a web app or designated web sites was considered. Security implications of this project and assess regulatory requirements was evaluated.
Air Gap Network: The current system has operated for over a decade it is reliable and provides the basic requirements of an editable information display dashboard along with the ability to upload documents through thumb stick USB drive. It is an example of a segmented network. However, it is old and unsupported. The thumb stick has potential for malware or mis use. It would be unacceptable to connect to network in its current form.
Azure Cloud has been proven across the company. It is not “air-gapped” but is fully secured by company IT department and Cyber Security teams as well as Microsoft as cloud provider.
Research extended to site visits to other radio stations and other units in the company. How their information is displayed and what could be incorporated into the design process was analysed. The information Display is just one component of it. Access to documents is another vital requirement. All users should have the ability to access the latest safety notices and changes to Work Instructions via PDF.
The new display should be similar enough to the old one such that a short briefing would suffice in training and navigation of pages intuitive.

12/8/2024		TMI	348	Friday 13th december								BREST 11 2300			SHANNON 02/0315		SUPER	UPPER			
F255 AND ABV												ETIKI 48N / UMLER			LANDFALL		F355 +ABV	F355 AND BEL			
RATSU 61N	127.850	A	ADARA	51/20	49/30	47/40	45/50	RAFIN				F360 & above	135.260		57N13W	AGORI	122.980	122.980			
LUSEN	133.680	B	RODEL	50/20	48/30	46/40	44/50	BOBTU	JAROM			F350 & below	129.505		57N15W	SUNOT	"	"			
ATSIX 60N	133.680	C	KOGAD	49/20	47/30	45/40	43/50	JEBBY	CARAC			SEPAL/BUNAV/SIVIR46N			563ON	BILTO	"	"			
ORTAV	133.680	D	ETIKI 48/1	48/20	46/30	44/40	42/50	42/60	DOVEY			F370 AND ABV	126.265		56N	PIKIL	"	"			
BALIX	132.730	E				42/40	38/50	34/60				F350 AND F360	132.025		553ON	ETARI	"	"			
ADO/ERA/ETIL	132.730											F340 AND BEL	124.675		55N	RESNO	133.360	133.360			
GOMUP	129.100														543ON	VENER	125.880	125.880			
LOW LEVEL	<F255														54N	DOGAL	"	"			
lus/atsix/ort	127.275											MADRID 11/2300			533ON	NEBIN	"	"			
BALIX/ETILO	127.275											F350 & above	135.955		53N	MALOT	131.150	131.150			
GOMUP	127.275											F340 & below	135.955		523ON	TOBOR	"	"			
															52N	LIMRI	120.040	120.040			
															513ON	ADARA	"	"			
SHANWICK (CTRL F6)			GANDER (F6)			ICELAND (SHIFT F6)			SANTA MARIA (ALT F6)						51N	DINIM	135.600	135.600			
13/1600			13/1525			13/0740			13/1041						503ON	RODEL	"	"			
															50N	SOMAX	"	"			
TC TF TI VF			VB XD XC			VD TD/XD			Pri: XE Sec:YE						493ON	KOGAD	"	"			
															49N	BEDRA	"	"			
18: TK TC TI			1830: XD VB			OVERFLIGHTS: 127.850			XE 11309						485020N	OMOKO	"	"			
19: TK TC QH						ICE LANDINGS: 126.550			YE 13354						484343N	TAMEL	135.230	135.230			
HF TO ATC'S			NEW YORK			MISC. INFORMATION									483839N	GELPO	"	"			
13/1600			DEC 12 2216			LPPO U03 TIL 18Z- VOLMET									Low Level		Below F245		124.7		
ORTAV + N 129.625						EGGX 11 TIL 1830 VOLMET									***CDO AM 123.950//EU 127.650***						
BALIX-LASNO: SEE GTL			VA TA												SCOT MIL 282.620//BIRD EMG VHF 119.7						
BICC: TF/VF LPAZ: VF/TF															CZQX SP +170 965 153 28						
LECM: VF/TF LFRR: TF/VF															UHF Vener + N 369.2 DOGAL + S 276.275						

Figure 1 ***The current layout***

Landing Page: [HDIP 24 Project Page]

The current layout will be the template. The layout will only display essential information. Tabs to miscellaneous and PDF and map to follow. The plan is to get a basic display rendered and get feedback from Users then.

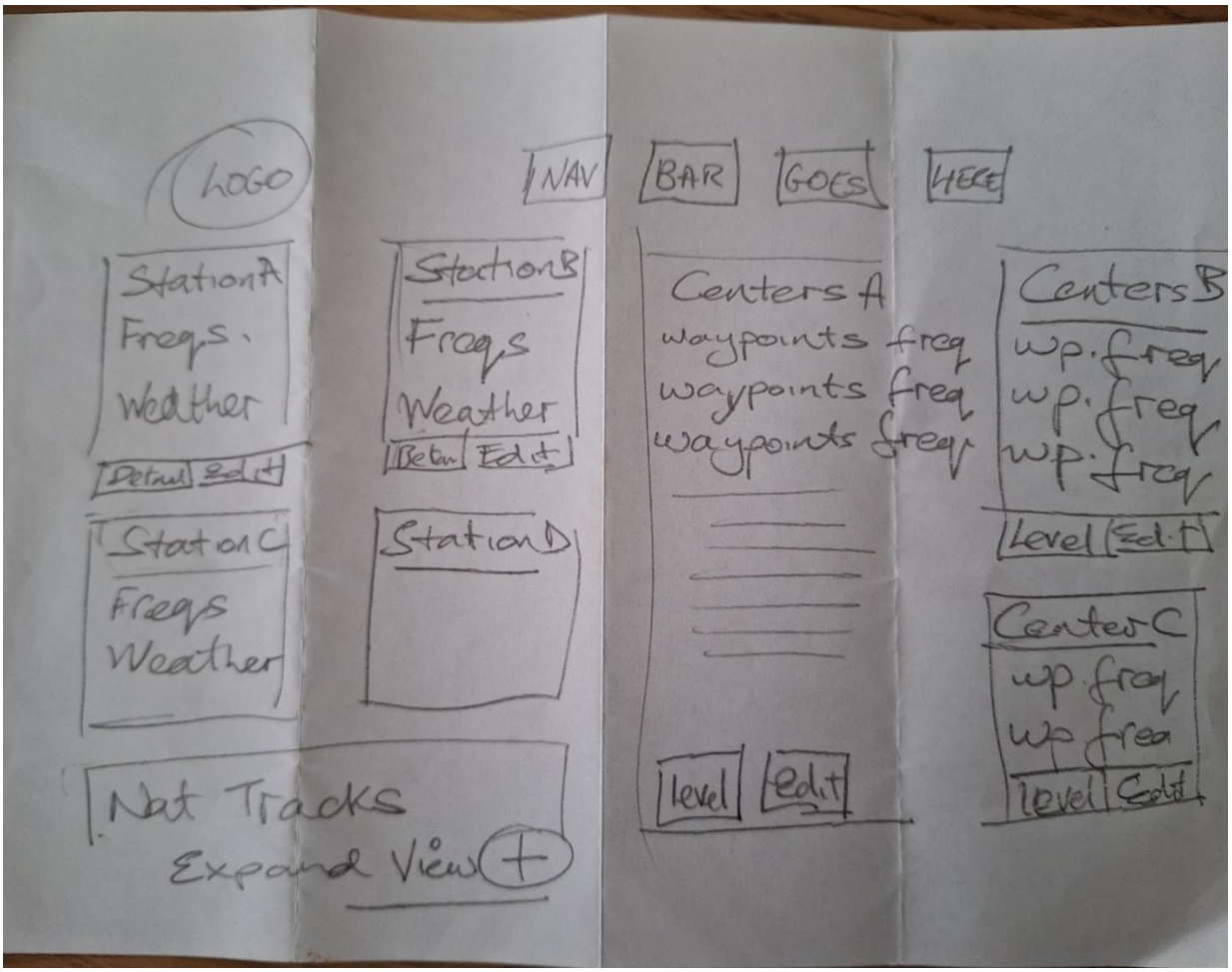

Figure 2 ***Sketching out Dashboard View***

Tech Stack Selection

The project was conceived with the HAPI (DATA BUILD AND VIEW) to SVELTE (MAP VIEW) migration. There were others considered like React, but the HAPI path had all the project requirements. The HAPI backend provided the Minimum Product viability. With Testing Joi Validation and Error Handling leading to API output which can be hooked to another frontend. I felt React and maybe Next.js were lead from the front and didn't have the same robust backend for the project foundations. By using HAPI to start I have a model that has been tested and is scalable for use with frontend.

MODELLING

User Stories

Persona 1: ADMIN

ROLE:

Technical Lead / Manager

GOAL

Manage structure: Create centers, stations, waypoints, frequencies.
Data integrity: Ensure data is valid, consistent
Control access: role based
Auditability: Explain to IT Engineering and Auditors

ISSUES

Confirm CRUD works, assess missing fields, seed structure, database

KEY FEATURES FOR ROLE

Log in with full privileges, CRUD, Formatting, Logs

Scenario (story):

The ADMIN logs in, opens the admin dashboard, adds a new Station with waypoints, and frequencies using the dedicated forms. The system validates the input, shows clear errors if something is wrong, and once saved, the new Station appears correctly on the dashboard for all users.

Persona 2: EDITOR

Role:

Co-Ordinator

Goals:

Edit frequencies, Nat Tracks and Weather Info on daily basis
Make changes without breaking the system
Report issues to Admin / Tech

Issues:

Ensuring changes survive dashboard view and later a map-based view
Avoid issues with referring to ADMIN for every small change

Key actions in system:

Edit existing data. Log issues with Admin / IT support

Scenario (story):

The EDITOR logs in, selects an existing FIR (Flight Information Region), updates a list of frequencies in the text area, and submits. The system validates the new values, highlights any malformed entries, and once corrected, the updated frequencies are reflected in the backend and on the map.

Persona 3: VIEWER

Role:

Operational Staff, Radio Officer or Trainee

Goals:

Easy reference to information and view on map
Trust the data is current and correct
Navigation: intuitive from Dashboard to Miscellaneous Information or Documents.

Issues:

Cluttered UI. Slow or confusing map interactions. Verifying Data.

Key actions in your system:

Access the dashboard (no login or read-only session).
Pan/zoom the map view.
Toggle layers (FIR boundaries, stations, waypoints, weather).

Scenario (story)

The VIEWER opens the dashboard, sees the Cesium globe with FIR boundaries overlaid, zooms into a region, and clicks on a station to see its name and frequencies. They don't need to know anything about the backend—just that the view is clear and responsive.

Initial DATA Model

ENTITY	ATTRIBUTES	RELATIONSHIP
Center	Id (Primary Key), name, code, waypoints (Foreign Key)	A Center has many waypoints (1:*)
Waypoint	name, (PK – unique), frequency, (future - latitude longitude), centerid (FK)	A Waypoint belongs to a center (1:1), A waypoint has many frequencies (1: *)
Frequency	Value (PK), type (level)	A Frequency (VHF) can be assigned to nil, one or many waypoints (0: *)
Station	Id (PK), name, code, frequency (HF), weather	A Station has nil to many weather (0: *)
Weather	Text	A weather report is associated with 1 station but there could be more than one (*:1)
Details	Id (PK), text, date, stationId (FK)	Child of station, a station has nil. One or several (0: *) A detail has one station (1:1)
Levels	Id (PK), text, date, centerId (FK)	Child of Center (0:*) A level has one center (1:1)
NatTracks	id, track number, date, list of tracks	Stand Alone but present in model

Assessing DATA Model

Waypoints are stable and very rarely change, so initially will not be assigned id. LAT, LON entry to be assessed as it is only a map requirement for phase 2. Frequency Primary Key is its value; it is defined by nothing else. Weather will just be a text entry. This may be re assessed after MVP.

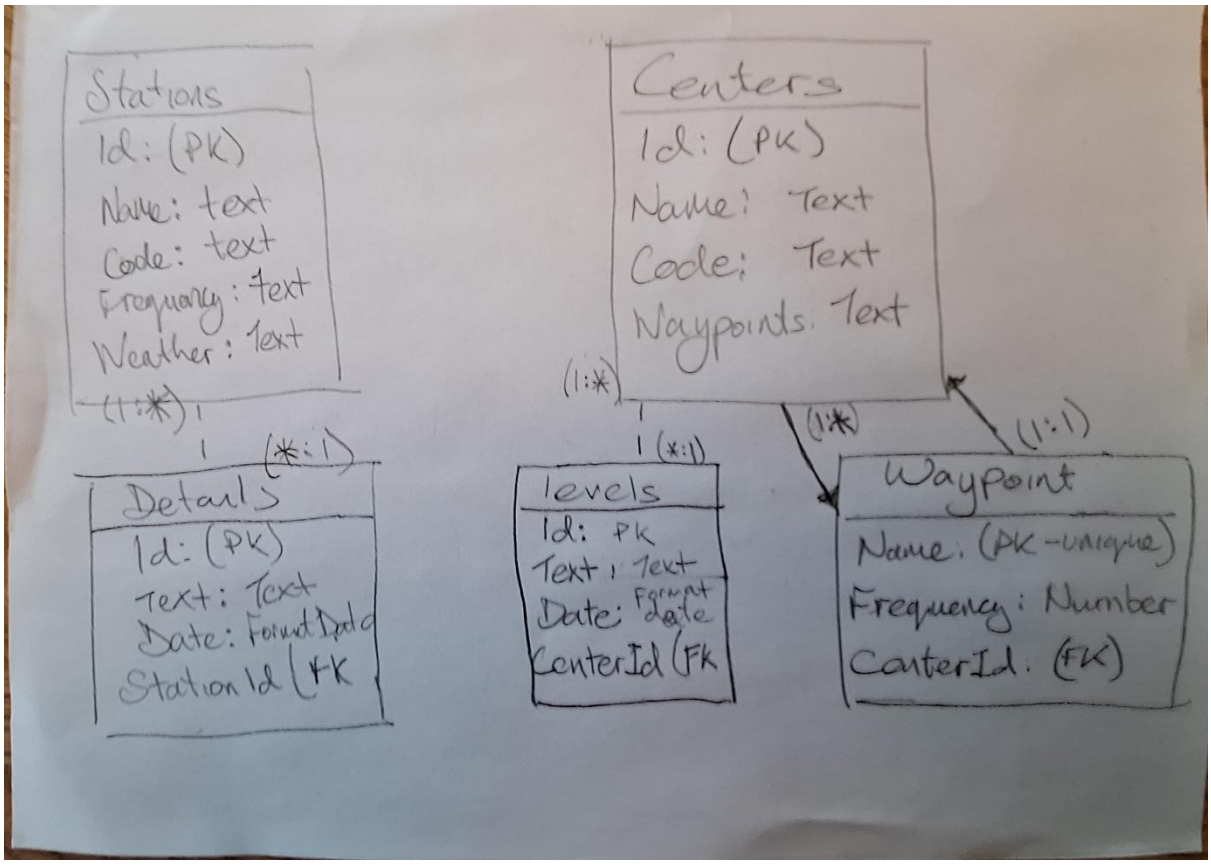

Figure 3 ***ER Diagram Sketch***

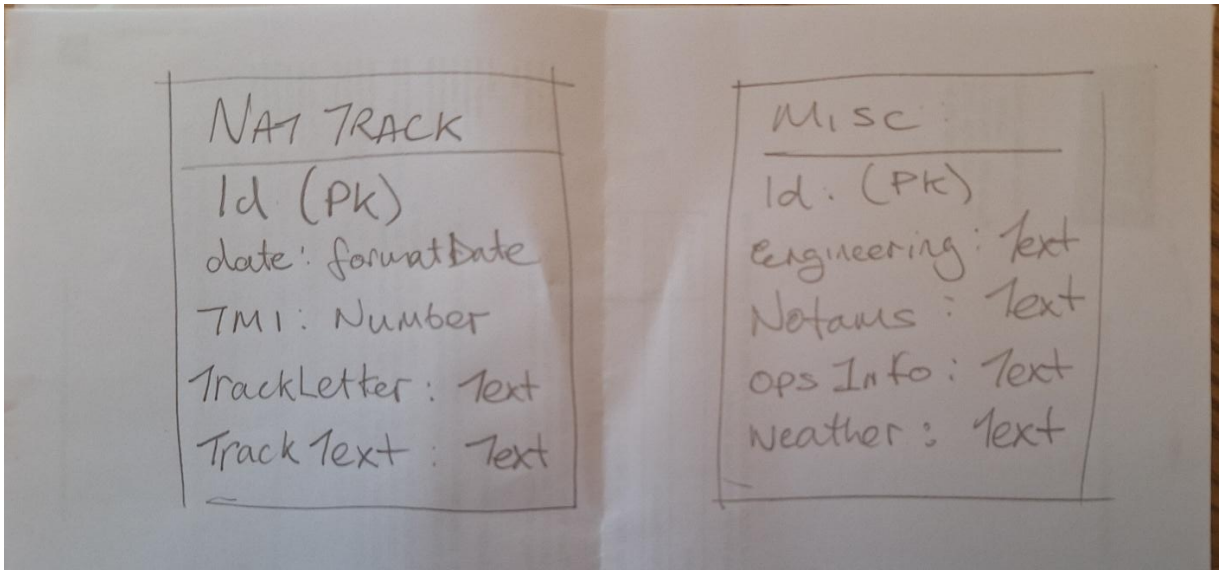

Figure 4 ***ER Diagram sketch continued***

The initial ER diagram is for MVP. The relationships are laid out as simple as possible. Once the initial build is complete then then feedback can be re integrated into the design process. Drifting in project scope is allowed up until interim (end Phase 1).

System Model:

Backend: HAPI

Input Data > Store Data > Display on Dashboard
Upload Documents/Files
Screen Size Adjustment
Role Based Architecture
Joi Validation
Session Cookie and BCrypt passwords
Expose Endpoints for Frontend

Frontend: SVELTE

Mount Map in Cesium 3D or Leaflet 2D
Stores Geo based data like FIR boundaries
Fetch API from HAPI backend
Fetch API from ADS B open sources

Future:

More integration with Maps
Add locations of Transmitter and Receiver Sites on Map to calculate Radio Coverage in distance.
Add Space Weather Data to calculate best HF Radio conditions
Weather and Nat Track Api fetch for dynamic output

Pipeline Model:

Admin:

Adds a station or a center, misc and Nat Tracks
Additional nested info for stations (details) or Centers (levels)
Seeds data by default (Can run a seed, or ADD entities)
uploads a file, role-based access, Error handling and JOI validation to avoid system failure.

Editor:

Edits a station or a center, misc and Nat Tracks
Edits child of Stations and Centers
Can delete out of date Files
Will monitor for Errors and Contact support
Will manage Map view in future

Viewer:

Reads Data comfortably
Navigates with ease
Sees role on Dashboard
May need update refresh alert!!

Wireframe

Main Dashboard: display of Frequencies and Weather and NAT Tracks.
Miscellaneous Page: for Operational and Engineering notes
Editing: Admin and Editor Update/Upload views
Documentation Page: PDF staff notices
Map Display to follow with successful Phase II.

IMPLEMETATION

Implementation Summary

The following is a list of what was achieved in this project.

PHASE I - HAPI BACKEND

- MVC for login / signup and authentication with hashing and salting
- Controlled number of users as per requirement.
- MVC for CRUD for Stations and Centers, Operational and Engineering Notes, Nat Tracks and Sigmet Weather
- Joi Validation for all inputs
- Error handling for null or inverse entries
- Documentation upload and store capacity
- Seeding for VHF Frequencies which can be amended as per air traffic control operations
- Model Testing for all CRUD and Seeding
- API Testing for all Json Data
- Role Based access for admin, editing and viewing
- Mongo Database
- Re Modelling and refactoring

PHASE II – SVELTE FRONTEND

- Cesium 3D view as a prototype and then Leaflet Map for 2D
- Fetch and backend integration of formatted and tested data.
- Fetch External API for Opensky and NASA
- API secret keys secure handling in dot env in HAPI
- ADSB data on map
- API information on frequencies and Weather from HAPI on the map
- Additional marker for VHF stations and VHF range mapped

Phase 1 - Minimum Viable Product (MVP)

Dashboard render of latest Station and Center Frequencies and Nat Tracks with timestamps. Document Upload capability from admin or editor pc and render at viewing position. Additional Information in Miscellaneous Tab which will refer maintenance issues, NOTAMS (Notices to Airmen), upcoming events affecting operation like ATC strikes or Space Weather or local weather warnings. Heads up on Military activity or unusual traffic scenarios like very northern tracks or Satellite service interruption.

Landing Page: HDIP 24 Project Page

Methodology

The methodology is to track the outline of the Full Stack I course. Establish solid HAPI backend and aim for frontend gloss with map output on svelte. First set up user route. Define users as the number of users will be limited to the unit and can be closely managed. The add station method will mimic the Web Development course in inception and follow on from that to update routing. It will incorporate a nested view to add notes via station id. This will be copied for Centers. Nat Tracks and Miscellaneous Info will be built similarly with no "Child" structure. An upload PDF route will be constructed in the vein of the image upload route in Full Stack I. Phase 2 will introduce a map view which will display ADS-B flight radar style live aircraft with some boundary polygons to start. API fetch calls will reference any of the Full Stack or Web Dev modules. The error handling and password policy will be based on the Security labs.

Planning

SPRINT 1 (WEEK 0 to Week 2)	**SPRINT 2** (WEEK 3 to Week 4)	**SPRINT 3** (WEEK 5 to Week 6)
Create *User, Station, Center, NAT Tracks and Misc. Info. Complete routes and navigation. Add unit Testing. Additional CRUD for child models. **Will start week 0*	Add password protections and input sanitisations, Joi Schema Validation Move on to seed, Mongoose Error Handling File Upload Discuss with Supervisor next move	Look to Role Based Architecture Add API routes Workshop with Colleagues Begin Svelte focus Interim Report

SPRINT 4 (WEEK 7 to Week 8)	**SPRINT 5** (WEEK 9 to Week 10)	**SPRINT 6** (WEEK 11 to Week 12)
Svelte Front End Fetch API Render to map **Week 7 lost to other commitments*	Code Review Report Review API calls Location markers on map Github Research for features	Showcase Week 11 Try more API (Maybe Weather) Review report, review code **Week11 lost to other Commitments*

Landing Page: HDIP 24 Project Page

Code Sprints

Backend

The focus was to get the Model View Controller (MVC) logic up and running and on display. The sign up and logging were initialised before the sprints. From then the model was built to establish the basic view below.

Figure 5 ***The initial layout after early MVC coding.***

The Interim Report provided an evaluation opportunity. The scope was re assessed. The model was refined. This involved a lot of refactoring and more emphasis on API and testing. This was reviewed and revisited throughout the timeline.

Frontend

Began early in the assignment just to define the type and amount of work involved. Initially Cesium Globe maps were used as base but switched to Leaflet as was more familiar and easier to use. The Data used in Svelte was derived from the Hapi backend or API calls. This is documented in the phase 2 Section.

GITHUB

The initial stage tracked the coursework with Git commits supported with simple commentary. Thereafter, milestone tags were used (e.g. Joi, Error Handling). When there was significant architecture integration (e.g. RBAC, Waypoint refractor, Sigmet CRUD), branches were used to resolve issues before merging with main. Branches are kept showing work attempted and achieved. There are two repos for each phase (IDS, IDSII). The use of branches and tags were more extensive in the backend. Appendix E refers.

NAT Track Route

By GIT commit 2.8 the following is achieved, CRUD, Sessions, Pages, Documents, RBAC and API. This follows the course work. The RBAC is documented in Appendix D. For the interim a code review was conducted. An audit of add routes failed. Nat Track Add functionality was incomplete. Matching the logic of the update Nat Track form proved to be difficult. The process of rendering Nat Tracks involved seeding data and updating the output. This was re assessed a a simpler text approach used. Researching some of the online Nat Track displays showed that the data is parsed from text as Nat Tracks messages change daily. The new Nat Track structure is to have the body of text input without nesting.
A problem arose from the way HTML renders text with an indent. The use of the pre-line (Mozilla, 2025) helped to end text on return and start neatly on next line. To keep this functionality repeatable, a partial textIndent.hbs ({{~text}}) was created and reused in Station and Weather to ensure uniform indent in text.

```
{{#if frequency}}
    <div class="box has-background-grey-lighter p-3 mb-3" style="border-radius: 6px;">
        {{> textIndent text=frequency}}
    </div>
{{/if}}
```

Figure 6 ***Eliminating white space***

API Testing

API Testing was conducted for API routes for Stations, Centers, Nat Track and User. Consideration was given to just trying the api.js files and hoping they worked. However, given the importance of this and evolving datasets, testing was implemented. API test routes were set up before fetching data from Svelte. The preference was to correctly format data in HAPI side to ensure smooth data flow. Initial testing for API User is outlined below:

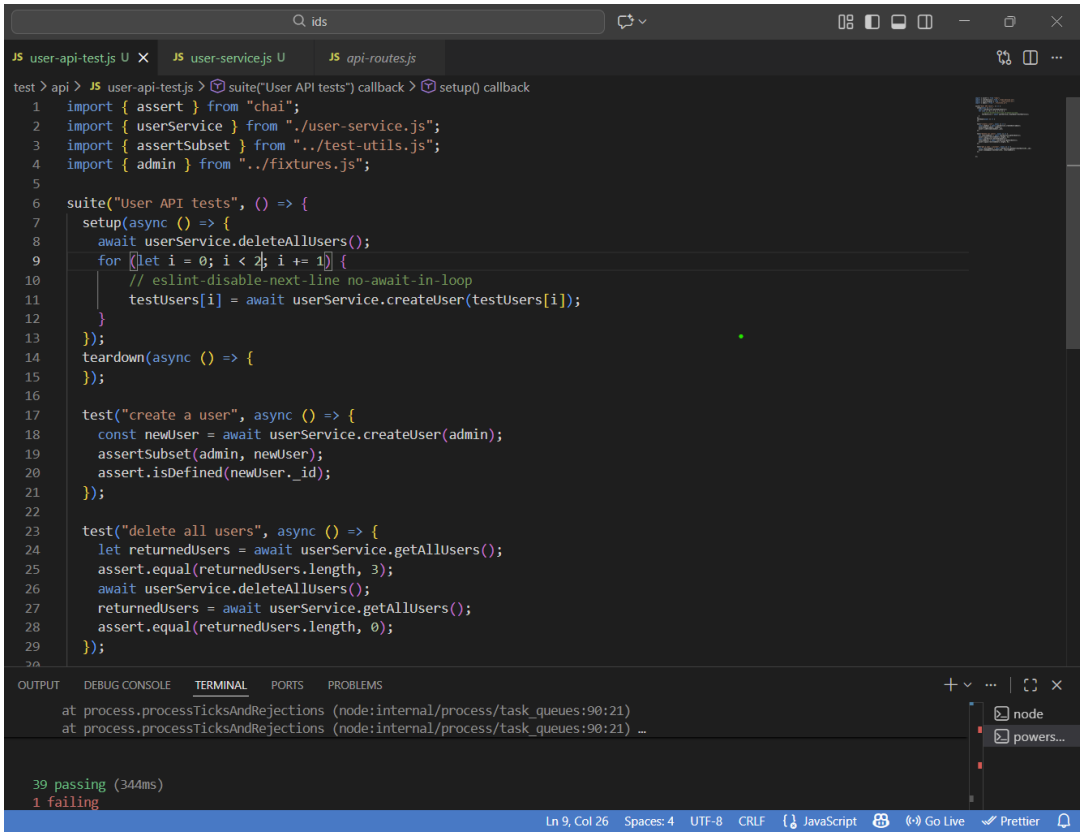

Figure 7 ***API User Test***

This had a failure before the create user was called. The service and user-api.js files were checked and there was a delete path there. It was noticed the delete routing from users in api-route.js was omitted. The test failed but succeeded in finding an issue. Testing resolved a lot of routing and dataset structure and misnaming issues (as below).
In this case there is a mismatch between /natTracks and /nats.

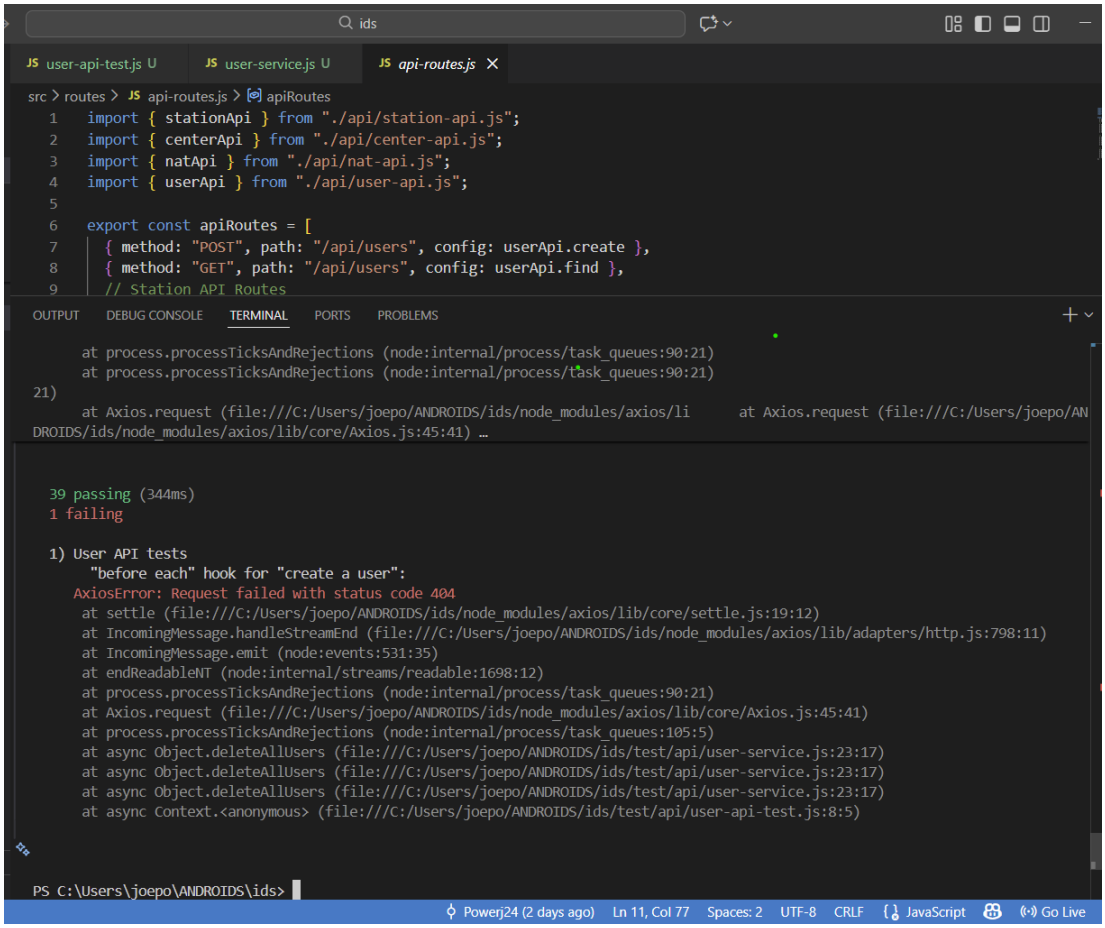

Figure 8 ***Test failure and routing issues revealed***

For Stations testing is confirmed following the same process as referenced in the Full Stack module.

Updated Model

Following the supervisor's advice, it was deemed best practise to document work more closely. This led to code review. It was noticed that the Center mapping and regex was brittle in structure. The course work and the relationship between Waypoints and Centers were reevaluated. As the Svelte App is to be populated by this as well, it is important to ensure data integrity. This data was not derived from solid architecture.
The key was to set out relationships in ER diagram. The initial assumption was that ATC Center was linked to waypoint. It didn't work. The plan involved redoing the logic of this with code and to use methods from the coursework. Now the Center > Sector > Waypoint dataflow was set out. Each ATC Center has Sectors and from Sectors is the direct relationship to Waypoints via Sector Frequencies.

Please refer to APPENDIX A for the WORKFLOW example on this. This substantial session is traced through the waypoints refactor and sandbox and vhf branches in GitHub.

The following is added to the ER Model:
Entity: Sector Attributes: id (PK), name, centerId (FK → Center.id),

Landing Page: [HDIP 24 Project Page]

Relationships: A Sector belongs to one Center (1:1), A Sector has many Waypoints (1: *)

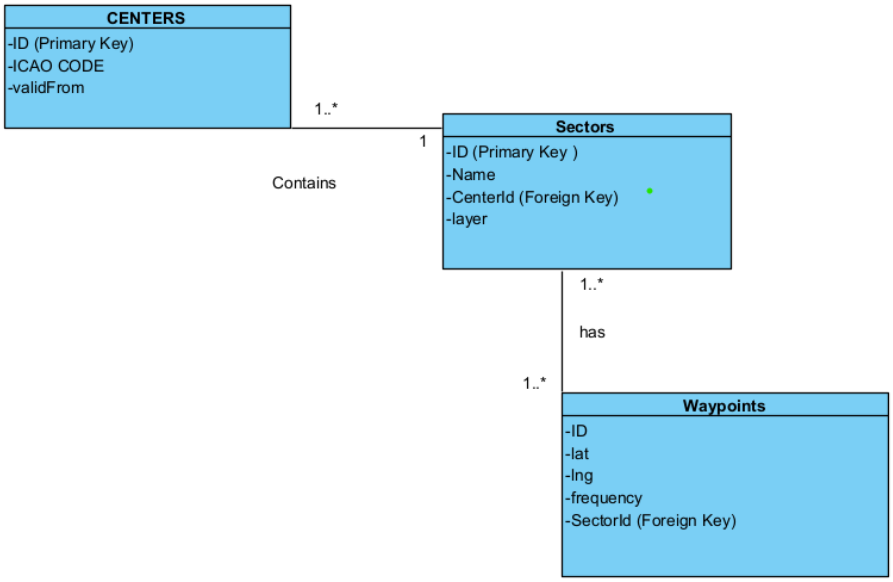

Figure 9 ***NEW ER model for Centers, Sectors, Waypoints***

This concept was extended and normalised further with frequencies introduced as an entity.

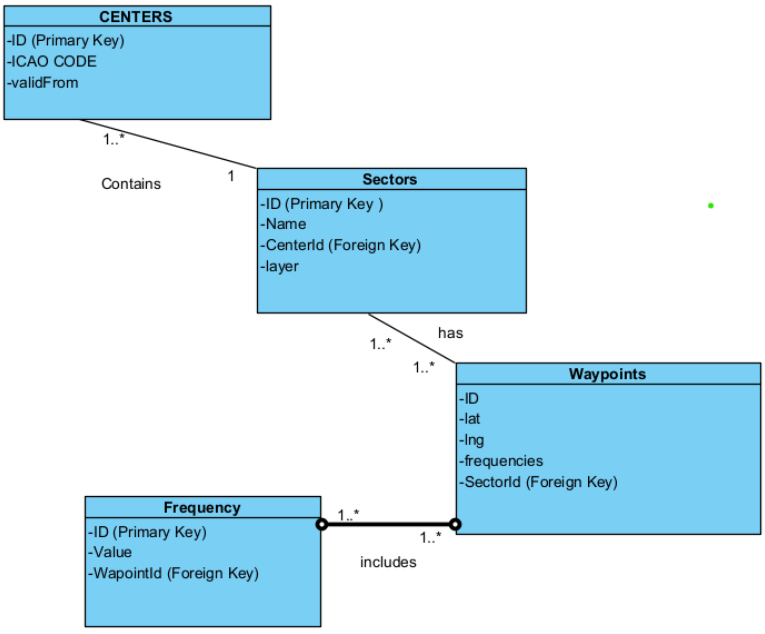

Figure 10 ***NEW UPDATED AND NORMALISED MODEL***

Sandbox

The issue of rendering updateable VHF frequencies for an ATC Center took over the project at the midterm point. The main dashboard looked completed, but the dataset did not match the

complex reality of ATC sectors and waypoints. This required significant overhaul of the model and painstaking refactoring. This became a project within a project and undoubtedly had an impact on deliverables for Phase 2.

The ER model (Figure 9) did not produce the desired result. (branch waypoints-refactor commits b0.17-0.19)
After normalising the model further as far as frequencies the result was just confusion. Even though the relationship held when trying to display waypoints by Center, three tier loop was needed. This was ok for render but then an additional element was required to update the frequencies, and this slowed the whole process down. It took possibly a week for this to get almost there but not quite. Again, an architectural decision was required, roll back to the 3-tier model (Figure 9) or continue with the 4-tier model (figure 10)
AN overlooked element when considering ER model was the direction of Flow. When I initially laid it out I had a bottom-up population model rather than a top down. I then tried to address this and normalised further with a frequency model. There was probably too much going on. I then accepted this github copilot mapping formula:

```
const centersWithWaypoints = centers.map(center => {
 const waypointsForCenter = waypoints.filter(wp =>
  wp.sectors.some(sec => sec.center._id.equals(center._id))
 );
```

I did roll back to commit b0.16 waypoints-refactor and try a top-down data flow but that didn't seem an obvious improvement.

Singular seeding was tried as per (Figure 11)

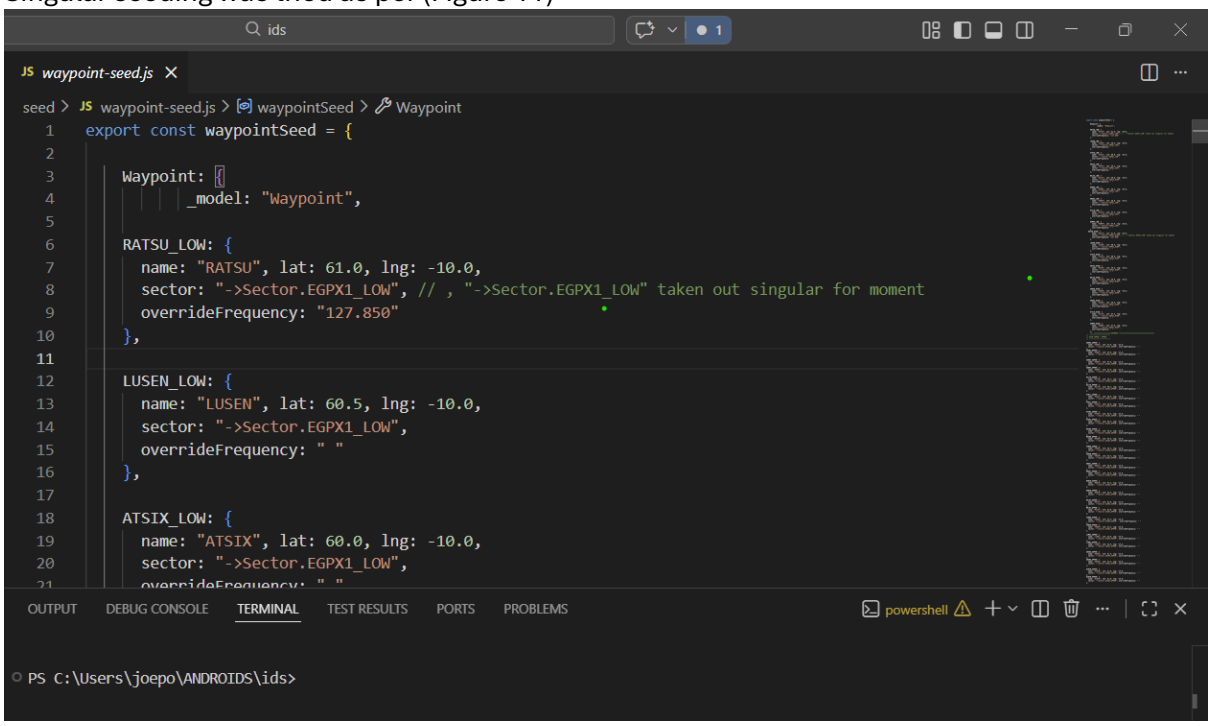

Figure 11 ***singular seeding each sector with associated waypoint***

Moving between models became complicated and confusing. A FINAL decision was made on the model. Sectors would hold default frequency and Waypoints could be updated by override frequency.

Landing Page: [HDIP 24 Project Page]

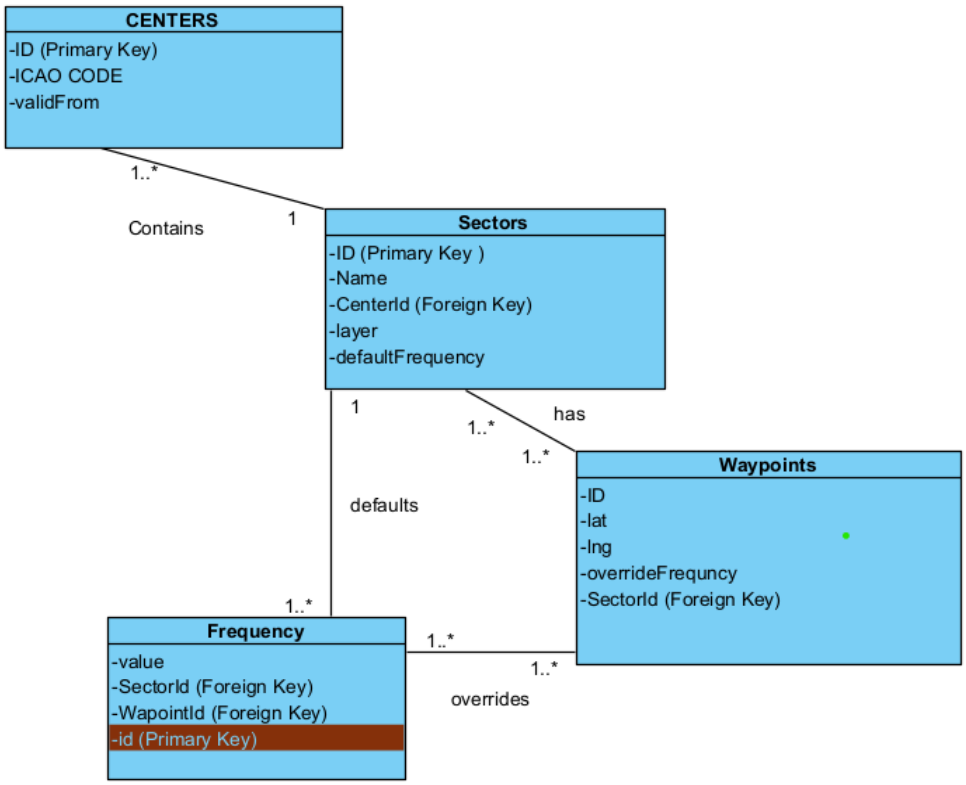

Figure 12 ***FINAL ER MODEL***

The return and render looked like this:

```
return {
  ...sector,
  activeFrequency: override ? override.value : sector.defaultFrequency,
  isOverride: !!override
 };
});
```

And in the view use this

```
<td>
 {{#if isOverride}}
  <span class="tag is-info">Override: {{activeFrequency}}</span>
 {{else}}
  <span class="tag is-success">Default: {{activeFrequency}}</span>
 {{/if}}
</td>
```

This is the model that will be followed as it will enable the project to get to a production level. The data was streamlined. All default frequencies will be seeded. Waypoints will be seeded without a null override Frequency then UPDATED to OVERIDE And RESET to DEFAULT. There is no delete route with this methodology.

Landing Page: [HDIP 24 Project Page]

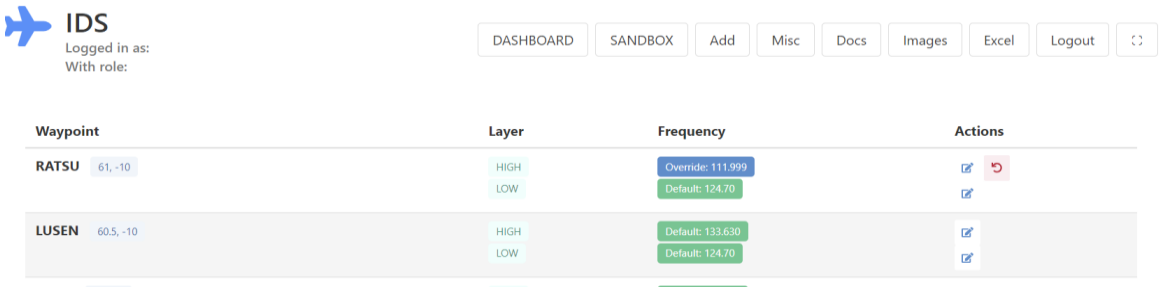

Figure 13 ***Override, default and reset works with final model***

GIT WORKFLOW SANDBOX

The work on this final model is outlined in b0.20 in waypoints refactor to SANDBOX branch. The TEST dashboard used in the previous commit were now refactored to SANDBOX. The simple model was then used. The frequency controller was finalised, and all /TEST files were either deleted or resolved to /sandbox.
All unnecessary files have been pared and cleaned up the ATC Center Controller to show new schema. This may lead to a loss of data (i.e. waypoint – frequencies flow from the original main) but it was not the required output for this project.
The following was achieved.

- THE Test to Sandbox migration is complete
- The new normalised model is in place
- The old waypoint-frequency parsing is eliminated
- The new Center Schema is in place
- New Seeds are available in Sandbox
- The frequency override and reset logic works
- The Sandbox UI is stable
- Test routes and controllers removed
- ATCController reflect new schema

The next stage is to get this data onto the main dashboard. The sandbox view was changed to VHF, a more operational label. The latest view of the VHF page

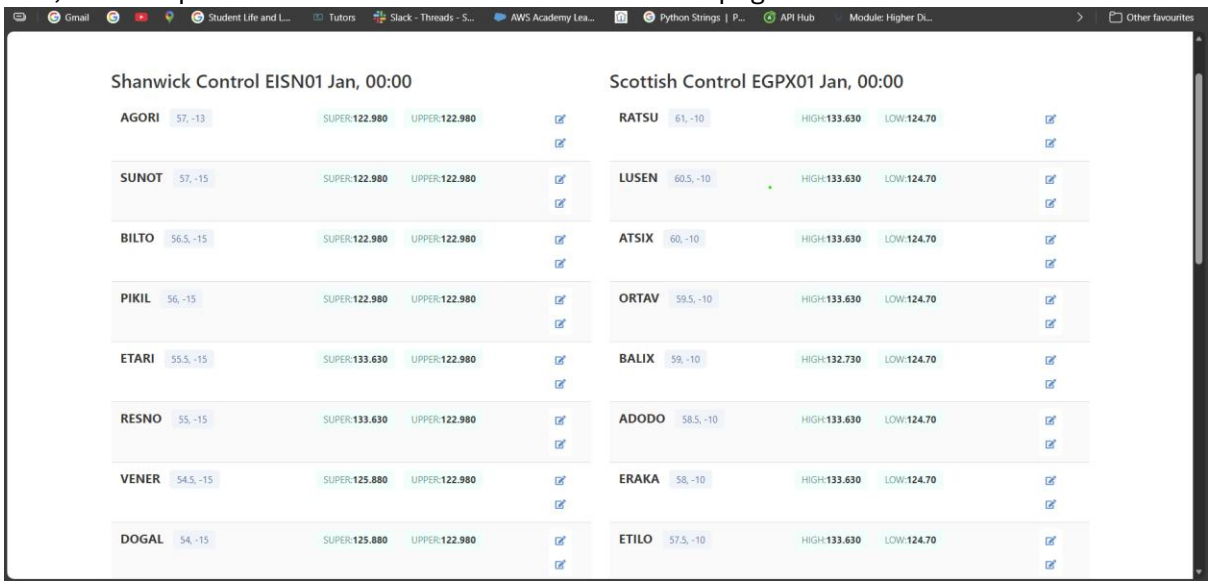

Figure 14 ***VHF Page view***

Revised Planning

Landing Page: [HDIP 24 Project Page]

The additional work involved in the VHF frequencies was did not match the original indicative 50:50 split between backend and frontend. The backend construction and refinement continued throughout. The Svelte part of the project was never ignored, and a simple map view and dashboard were introduced on time as a milestone.

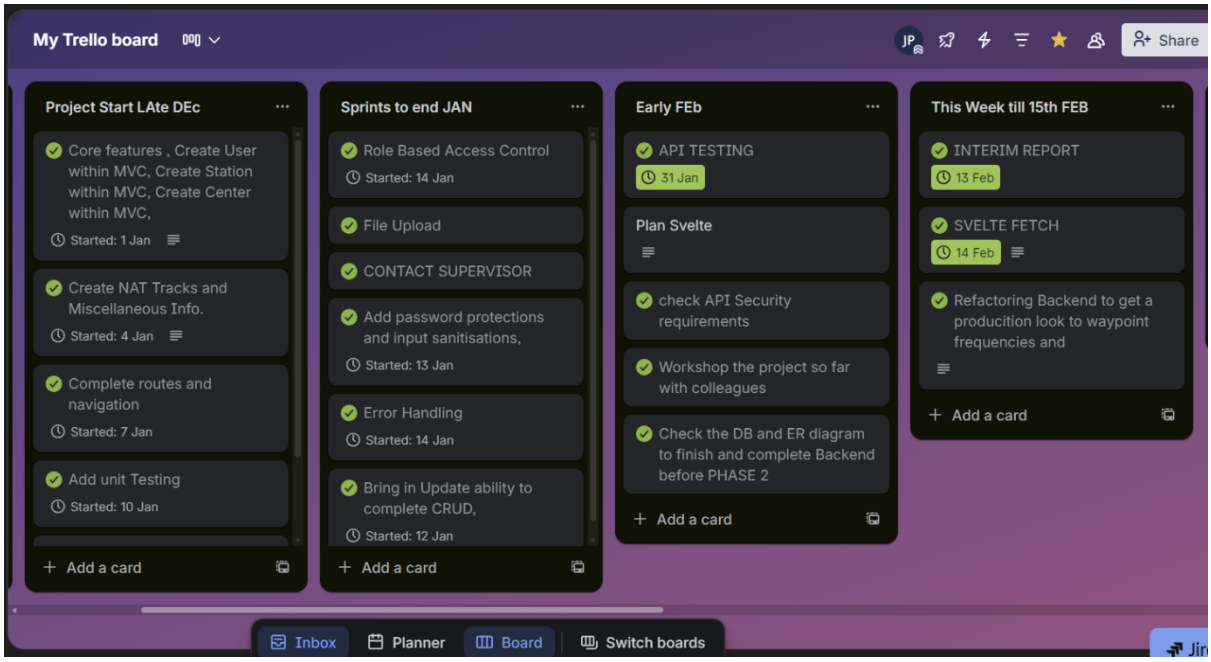

Figure 15 ***FEB 11th Trello board***

The above board does not consider the excessive work gone into reworking the VHF frequency layout. This has taken over 2 weeks from the point of ER Diagram to a stage where the VHF frequencies are listed in an agreeable production form. When this board was written, there is still a need for API testing RBAC and error handling on the significant waypoint refactoring.

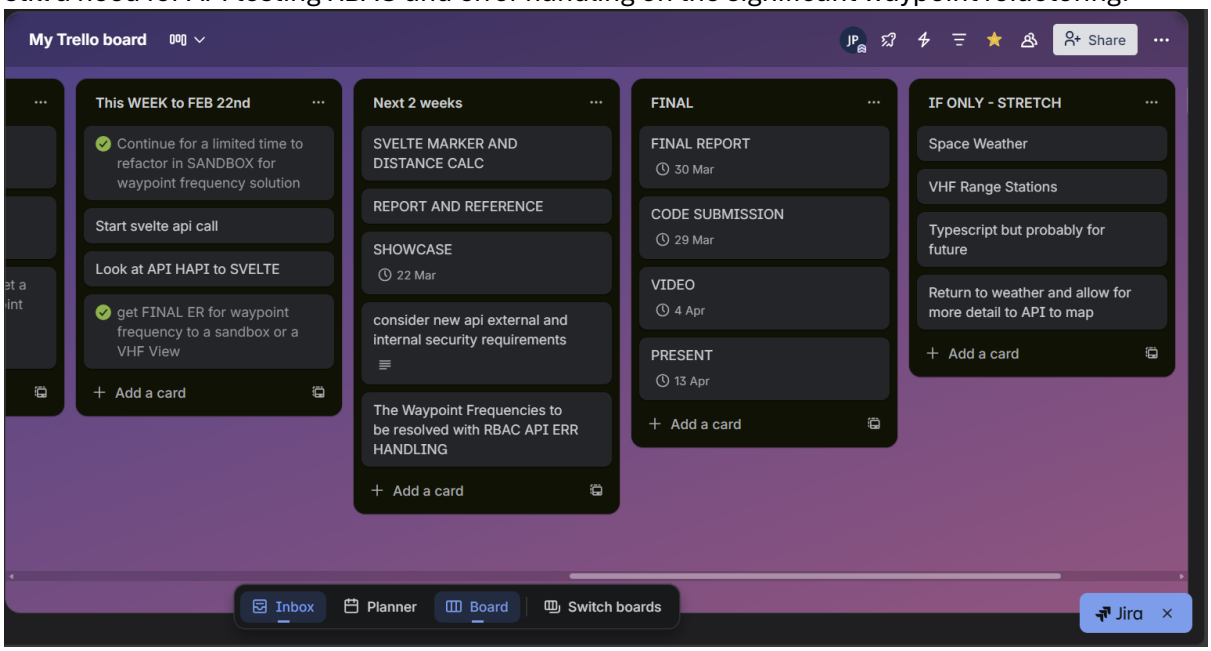

Figure 16 ***Trello Board FEB 20th***

RBAC For New Model

Landing Page: [HDIP 24 Project Page]

Frequency Controller

First import {allow} for access.js which lays out what role has what permissions. Instead of extensive middleware, the link file was compressed as seen below (Webdevsimplified, 2021)

```
export const access = {
    admin: ["add", "edit", "upload", "delete", "view"],
    manager: ["edit", "view", "upload"],
    editor: ["edit", "view"],
    viewer: ["view"]
};

export function allow(user, action) {
    return access[user.role]?.includes(action);
}
```

Figure 17 ***Setting out Roles and access to deliver RBAC***

Add the following line inside the controller and apply to functions that require action.
const user = request.auth.credentials; if (!allow(user, "edit")){

    return h.response("Access Denied").code(403);

  }

This sets out that only the users with edit role can edit.
For handlebars to render the correct RBAC, use the following helper

vhf-card.hbs | vhf-dashboard-controller.js M | frequency-controller.js M | server.js

src > server.js > init > Handlebars.registerHelper("hasRole") callback

```
28  async function init() {
120
121   Handlebars.registerHelper("hasRole", (user, role, options) => {
122     try {
123
124       if (!user || !user.role) {
125
126       return options.inverse(this);
127        }
128
129       if (allow(user, role)) {
130         return options.fn(this);
131       }
132       return options.inverse(this);
133     }
134     catch (error) {
135       console.error("Error in hasRole helper:", error);
136       return false;
137     }
138   });
139
```

Figure 18 ***RBAC Helper***

This helper “hasRole” declares allow function which relies on user and role to apply rbac in handlebars and if not available will not apply rbac to the view. Applying RBAC to hbs below:

```
</span>
{{#hasRole ../user "edit"}}
<a class="button is-white is-small px-2"
   href="/vhf/frequencies/select?waypointId={{../_id}}&sectorId={{_id}}">
  <i class="fas fa-edit has-text-info"></i>
</a>

{{#if isOverride}}
  <a class="button is-danger is-light is-small px-2 ml-1"
     href="/vhf/frequencies/reset?waypointId={{../_id}}&sectorId={{_id}}">
    <i class="fas fa-undo"></i>
  </a>
{{/if}}
  {{/hasRole}}
</div>
```

Figure 19 ***Applying RBAC to handlebars***

Use the helper to match user with role. It is important to set out the level of the user. In this case the level needs to come down two more to ../../../user to run as it is in a sector loop with a waypoint loop within a group loop. This works for the moment but looks fragile to me long term. The RBAC did not apply uniformly but there was no LECM data seeded so the other group's view failed in total. All in all, the RBAC did not work for the grouped logic applied to the controller to render these vhf frequencies. It was the main borrowing of copilot code (SEE APPENDIX C) that was used in this project.
The solution to the unwieldy nested grouping vhf display was to apply RBAC to the page itself. The VHF dashboard is kept and the reset frequency works. The view then is copied into the main dashboard which is available to all.

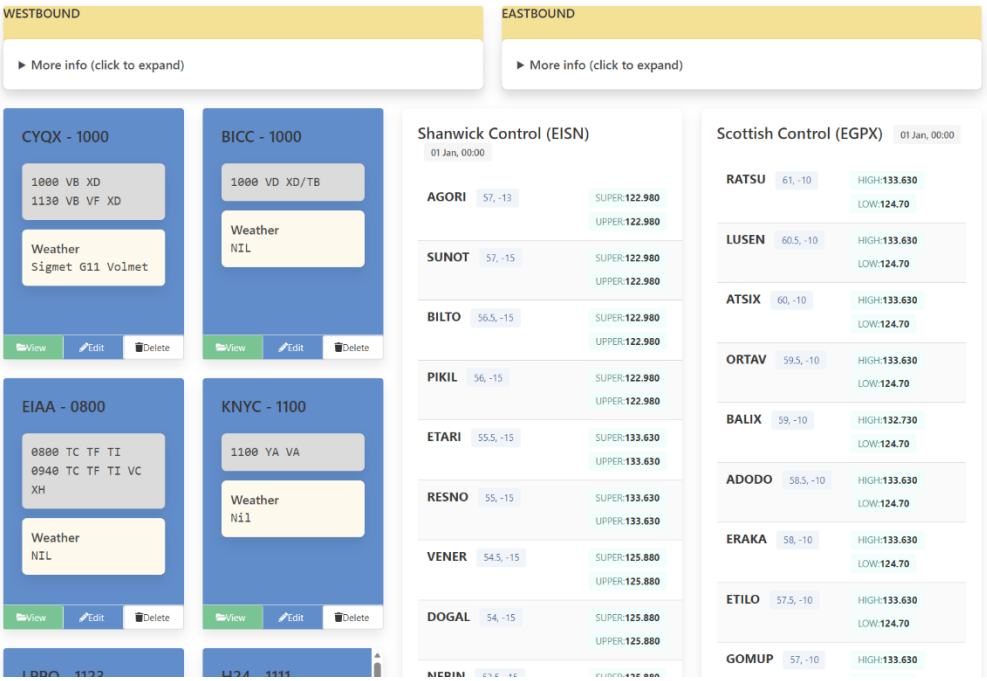

Figure 20 ***The Demo view of the Dashboard***

PHASE 2 – Map View

The pipeline of the project allows for Phase 1 (MVP) to stand alone in a local environment. The purpose of Phase 2 is to expand the visual element of this project. The data rendered can also

work within a map view. HAPI fetched data can be viewed beside sigmet weather, space weather, VHF range and aircraft in flight. The ADS B (sudu RADAR) would be a visual aide and a dynamic addition to the application.

Svelte was initiated with the guide delivered within the course. Apart from this the reference points are the Smillex(2022) GitHub, a substantial adsb project.

Set up svelte
npm create svelte@latest idsii
cd idsii
npm install

At this stage it was confirmed to stay in JavaScript. TypeScript can be added later.

Src/routes/+page.svelte is the landing page.

For Cesium

Npm cesium.

Routes/map/+page.svelte for map page.

For the basic Map (Cesium Community, 2020) and for rendering a polygon was used to complement the main reference ISmillez (2022) GitHub page. It was set up with the basic Cesium JS, so no API key was needed and is offline compatible. Cesium must also be declared in the vite.config file (Cesium Community, 2023) to ensure assets and runtime work in the build.

API

API routing had been established. However, the major re structure of data via population rather than CRUD also affected by this. This needed to be updated for centers at least so that Svelte could read that data.

The api files were redone for the new model with seeding in mind. Testing was unsuccessful but resolved when it was noticed that the testing ran through the old seed.js file and not the new vhf seed

The api testing is the last line in the backend for now. An attempt to run seed api testing did not seem to line up. It is a server issue judging from 503 response to testing. The server will be left alone for now. It would be preferable to get a confirm data before Svelte fetch. However, there was a few issues with the seed waypoints that needed a correction. Alternative now is just to get json output on svelte.

Fetch

While Phase One was well under way, A start was made on this. Just to see if it would work. The Svelte was initiated as per docs. The main page view was set up to be CESIUM Map.
There were markers for waypoints manually inserted.

Landing Page: HDIP 24 Project Page

Api fetch for stations was derived from

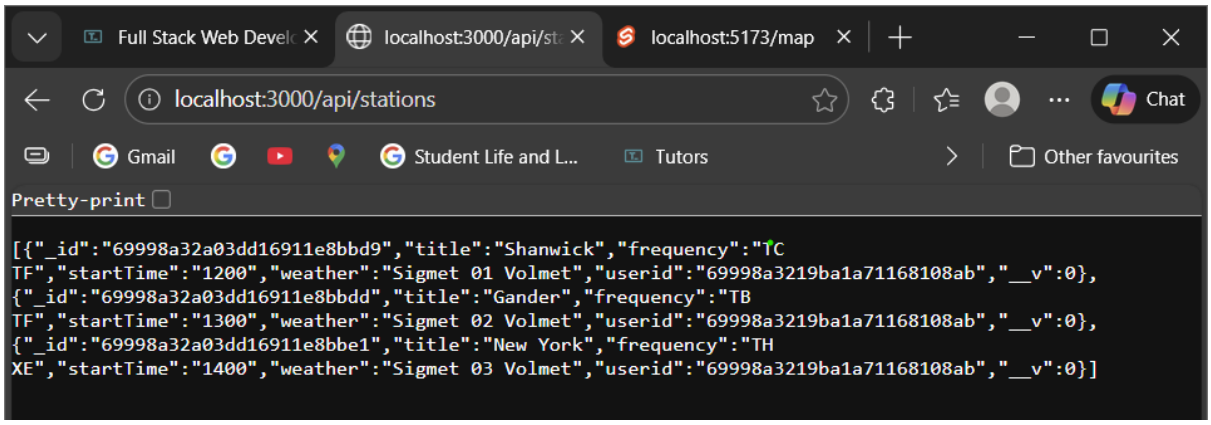

Figure 21 ***Json data exist for station***

Moving on in same vein to other datasets
DASHBOARD added. The login page is omitted to prove concept.

It's a crude look without much styling other than ICON. The station API fetch works but the other seed only datasets FAIL just like they did in API Testing.

After much head-scratching there was the sudden realisation that the new model had a routing structure via /vhf as per vhf-routes file. That was not it. I revised the tests and got get and find mixed up to correctly reach functions in store. Also asserted default frequency instead of waypoint in Sector

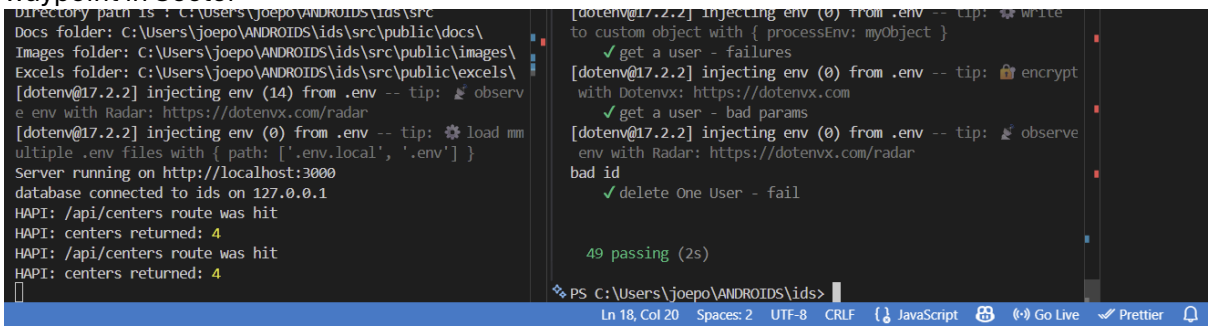

Figure 22 ***API Tests finally pass***

Leaflet

Cesium uses up a large amount of GPU even just in the view. I had considered the Globe view as being good for aircraft in elevation but not for terrestrial view. Leaflet was used to populate with API data. It has been requested in workshops for this type of view.

Install leaflet, it had some vulnerabilities but referred to eslint.

Landing Page: HDIP 24 Project Page

Set up Leaflet component page and build from there

All API data now available and referenced. The json form is parked on a Page for each. For waypoints the data was resolved from json to readable and then added to the map view. Still in JavaScript.

```
L.marker([lat, lon])
 .addTo(map)
 .bindPopup(waypoint.name);
```

The marker is unwieldy looking for what is required.
There is an improvement is using (Leaflet Docs, 2010-2025)

```
 L.tooltip([wp.lat, wp.lng])
     .setContent(wp.name)
     .addTo(imap)
     .bindPopup(wp.name);
   }
```

This works better but the most important thing is to prove the pipeline works. Data is available on the map.
Nav bar was also resolved. The view shows data and Globe and Map view. The login route was bypassed. The leaflet map now is the focal point of Phase 2. Waypoints have been rendered and other API data will follow that path.

Expanding out the features of the map to include markers for VHF stations and boundaries. These were then added to existing base layer using layer group (Leaflet Layer Group, 2010-25)

Opensky

The Svelte page doesn't apply session and will only run through the HAPI App. FOR this reason a public API was used to retrieve data for ADSB from Opensky. The data has returned in JSON form and now needs to render onto the page as per Waypoints. There is an importance to apply a boundary the aircraft per latitude and longitude of the region (Eastern North Atlantic).
Using the open sky (OpenSky Rest API, 2021)

```
`https://opensky-network.org/api/states/all?lamin=${lamin}&lomin=${lomin}&lamax=${lamax}&lomax=${lomax}`;
```

Setting the area to the parameters of the oceanic airspace gives back only some data. There is no radar coverage over the Atlantic, so the model relies on ADSB satellite for accuracy. This current basic model demonstrates the use of the data but to provide a de facto RADAR over the north Atlantic a satellite constellation must be accessed.

Space Weather

Using the same API pathway, a dataset for space weather was retrieved. The API key is kept in HAPI dot env file and routed from the server.js file. The first one attempted was the CME (Coronal Mass Ejection) data. This is a very large file of Solar activity. With no plan on how to distil this information, was research was required to best use this NASA API (NASA, 2025).

I was reminded of the use of POSTMAN (Andys Tech Tutorials, 2022) to refine the parameters of an API call. The calls could be targeted to a latest update or set as an alert if certain thresholds for solar activity were breached.

When setting an API key to the dot env file it is essential to log in to the HAPI backend with correct credentials otherwise the APT call will fail. Initial data was sought for CME, Geomagnetic Storm (GST) and Solar flares (FLR). These would indicate high solar activity that would affect HF radio conditions. Vital to know but difficult to express on a map. The option of a Chart could be used. For GST is there is no event there is no data. The Aurora Display of January 2026 is an example where this is very active.

The API call clearly needed to incorporate a latest issue or just use error handling to prevent the app from crashing. The absence of a login would lead to a 503 error and a HTML return error. The absence of data is common for space weather, where an event may have not taken place or has not been added today. There is also rate limit to the data drawn down. In this case the latest or should be just updated each day rather than constant or continuous API call for Space Weather.

SIGMET Weather

SIGMET (see Glossary) Weather was prioritised over Space Weather as a key deliverable.

The Sigmet Weather information would normally be derived from forecast and Pilot reports (PIREPs). It would consist of weather phenomena ranging from Thunderstorms to turbulence to Volcanic Ash. This is bound by an area or line and a range of Flight level. This is a more beneficial to the user as a map element than text stream. The User can then orientate aircraft position versus weather event.
After failing to model Nat Tracks as desired, a similar but more complex data structure to Sigmets, another attempt was made to draw lines on a map. This time the base reference was geoJSON. Which is a mapping standard, to derive JSON from geometric data.
For an earlier attempt at Nat Track function for a form to add Points. I cound not get this to work in part due to text waypoints which are required. I amended this method (Web3Schools, 199-2026) to to be used in Sigmet building.

```
<!DOCTYPE html>
<html>
<body>
<ul id="myNat"></ul>
<button onclick="addPoints()">Add Points</button>
<script>
function addPoints() {
  const lon = -20;
  let lat = 55;
  for (let i = 0; i < 4; i++) {
    const li = document.createElement("li");
    li.textContent = ` ${lon}, ${lat}` ;
    document.getElementById("myNat").appendChild(li);
     lon += 10.  }}
</script>
</body>
</html>
```

Landing Page: HDIP 24 Project Page

The goal is to create eventually geoJSON (Mansoor, 2024)

```
"geometry": {  "type": "Polygon",
      "coordinates": [
       [  [-79.4665, 43.6465], [-79.4641, 43.6515], [-79.4573, 43.6510], [-79.4571, 43.6460],
        [-79.4665, 43.6465] ]  ]  } },
```

NAT and SIGMETS are transient and therefore need to be created and deleted but not updated.

Accessing Svelte

The objective at the start was to run svelte in typescript as per coursework. This would involve setting up login routes for that application as well. The expectation is that this will take some time. For this project considering deployment is not the immediate goal. The Svelte app will run parallel to HAPI and API sources secured there. A branch svelte login begins the authentication and typescript pages but left outside main branch for now.

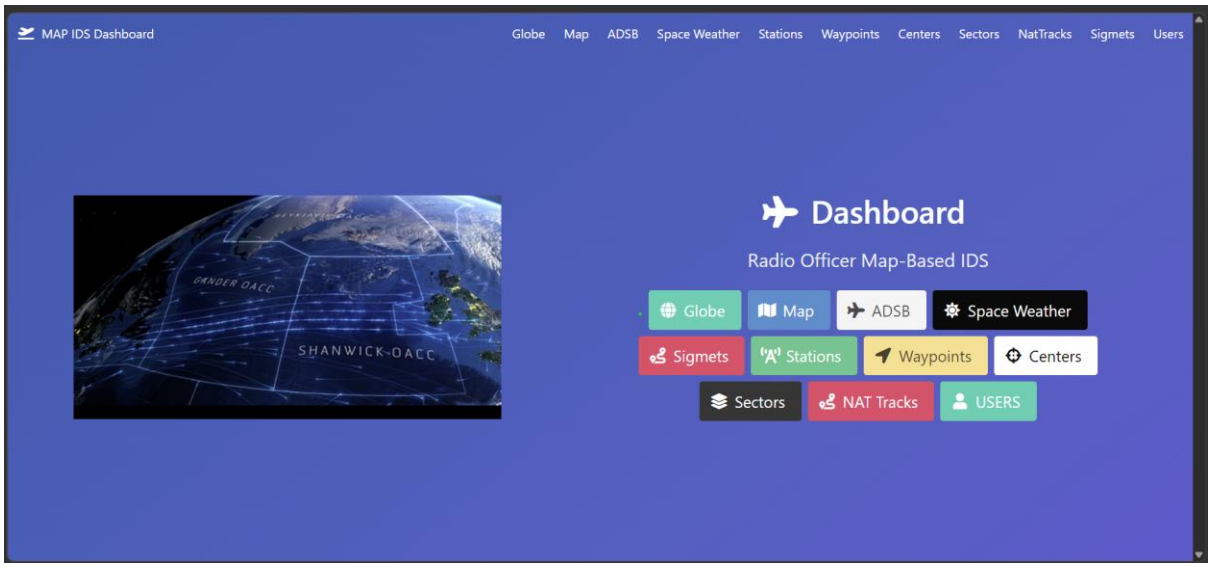

Figure 23 ***Data Dashboard for all API***

The Dashboard for the Svelte application is good but really it will need to be a map view and all data transposed there. A direct link to the map view is on the final HAPI dashboard.

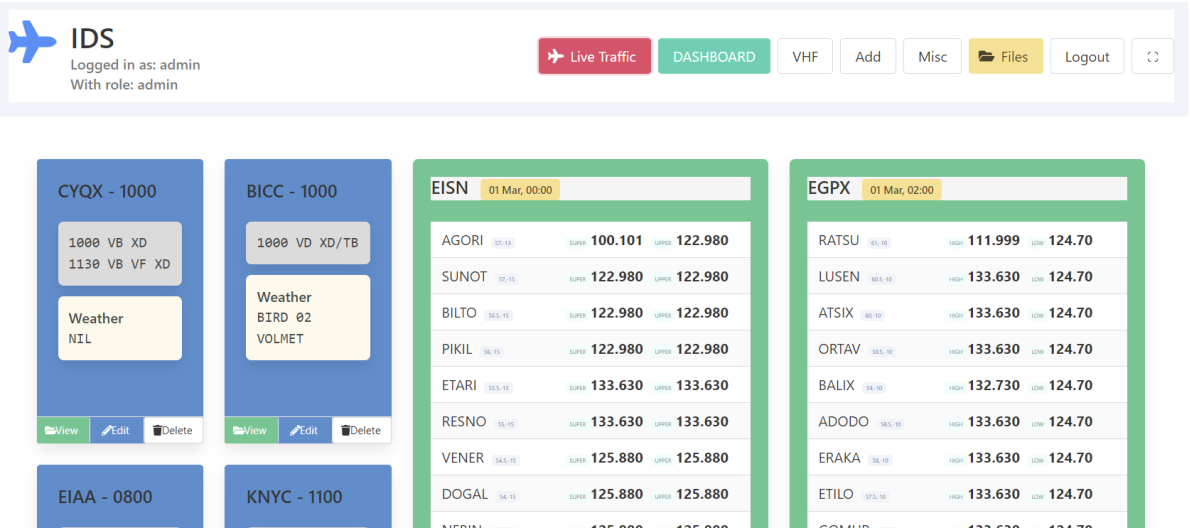

Figure 24 ***Final Dashboard with Navbar***

EVALUATION

Interim Review

Meeting with Supervisor

On January 28th a first meeting with the project supervisor was conducted. Progress was steady until this time but needed some direction now as phase one moves to phase two. The supervisor advised that documenting everything with references and any interaction with an AI was necessary. This was not something done with dedication up until this point.
The majority of the first sprints were tethered to the course material. Where something different like RBAC was performed it was tagged and noted accordingly in Github, but not extensively elsewhere. From now on the project will have more logging and documenting.
The main backend coding block is almost complete. There is a renewed determination to get code associated with the coursework refactored and adapted as soon as possible.
On the 28th of January the project Trello board indicates the backend is almost there and from now there is the frontend and some stretch features associated with the frontend.

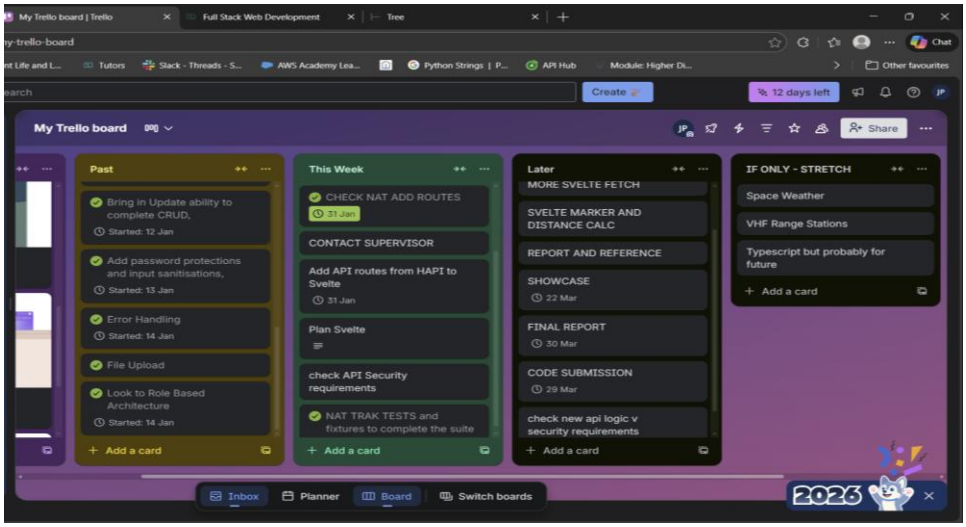

Figure 25 ***TRELLO BOARD at 28th of January***

Landing Page: [HDIP 24 Project Page]

Phase 2 code was concept was derived from a GitHub page (Smillex,2022). This reference was explored ahead of schedule as a roadmap.

Code Review

The initial CODE sprints were completed with the intension of getting to the MVP as soon as possible. Following discussion with the Supervisor, it was decided to commit to a Data Review to ensure that the code is verified and sourced correctly, if not, amended. This is reflected in commits at 2.8 level and are referred to in the appendix. It also led to the significant task of remodelling and refactoring.

Personal Review

The main personnel learning is perseverance. Constant adherence to the fundamentals of the course work were crucial. Mistakes or skill gaps needed to be overcome. This was done revisiting the coursework and going again. This led to improved modelling or simplifying code (RBAC, see also Appendix D).

Architecture

There was a re-evaluation mid-way through the project. The initial modelling of waypoint and vhf frequencies was both simplistic in concept as it did not account for real world variations in data structure. At the same time poor in execution with the use of complex regex to render.
With the supervisor's advice, it was understood to be more disciplined in planning and execution. A decision was made to to rip up some of the modelling and follow closely the style of the Full Stack Module by using referenced model and data population techniques. This took a long time (20 plus commits) over two weeks to come back to a satisfactory outcome. This was definitive.
The updated modelling helped the solution. There are now four referenced entities in the model. Center > Sectors > Waypoints > Frequencies. A decision to require only CRUD on Centers and Frequencies was crucial as Sectors and Waypoints are constant. There was a clash between Sector and Waypoint frequencies. This was resolved by applying default to Sector and override to Waypoint. The active frequency would be the published frequency. A 2-week process, but difficult to see until the 4-tiered model was described.
Generally, Architecture was very exploratory but very rewarding. Making sense of the outcome and how to get there based on experiential learning.
One of the last commits was the SECTOR branch which proves that this model will work for creation of waypoints correctly. This was not achievable with the old model. This branch is left outside the main repo as it is not required for the demonstration.

Coding

Coding performance was fine within the guard rails of course work. Using referenced assistance new challenges were met like RBAC (appendix D) and Helpers. The MVC structures were built efficiently. The experience of the lab work made understanding of expected errors and debugging requirements very familiar. Issues like no seeding, database integrity, misnamed functions were handled with composure. Generally debugging through console gave an indication of where the code or route or store was deficient. For example, the following method was used to pick out an id when rendering was lost to loop hierarchy. This worked really well when debugging CRUD routes.

```
{{#each excels}}
  <div class="box">
    <a href="/static/excels/{{this.filename}}" target="_blank">{{this.filename}}</a>
  </div>
  {{#hasRole @root.user "delete"}}

  <p>this id: {{this._id}}</p>
  <p>that id: {{_id}}</p>
  <p>other id: {{../_id}}</p>

  <footer class="card-footer">
      <form action="/excels/delete/{{../_id}}" method="POST" class="card-footer-item">
        <button class="button is-danger is-light is-fullwidth" type="submit"> <sp
      </form>
    </footer>
    {{/hasRole}}
{{/each}}
</div>
```

Figure 26 ***This, That. Other – finding id hierarchy***

Use of AI

The words in this report are all my own or at least referenced with honesty. There is absolutely no vibe coding in the submitted repo.
This project is based on the coursework and tutorials of the CS HDIP programme in SETU. I expect that 90 % of code and architecture is sourced from this course, especially the Full Stack I module. This module was accessed extensively. The rest comprising of referenced code from websites or docs or at in specific cases an AI agent.

The data structure for parts of this project is quite niche so there was no real reservoir to draw on or copy in terms of data structure. All architecture decisions were made independent of AI. Copilot was used sometimes a sounding board, but it didn't contribute to the outcome. I did follow AI advise but to a dead end. (branched -b0.16 waypoints refactor). This was a waste of time and made me redouble efforts to get a resolution to my data structure.

This was important to control the flow and scope of the project.

There was assistance with rendering data to the page. For instance, some of the data used was grouped using a 4-tier loop which was resolved with copilot assistance. IDE suggestions accepted if made sense to the overall scope and resolved inaccuracies in coding attempts.
No copy and paste was used unless to check for <divs> etc. however, this practise was eliminated upon a code review as it could lead to erroneous inclusions or indeed exclusions.

As such the expectation is that there are no surprises submitted and all can be verified and understood well enough to fix or amend.

Refer to Appendix for more.

Timeline

The plan v actual up to week 10 started to align. Most of the requirements for the project submission had been attempted or completed. The initial code sprints were done in a dash to get a basic MVP but did not assign enough value to the planning or eventual outcome. After discussing with the supervisor, a more disciplined approach was made. Deleting of unverified or referenced code and re-structuring of models to a pause on the initial progress. This was for the better. Once the MVP was achieved, progress was on the Svelte component of the project. Changing from Cesium to Leaflet was led to a catching up on the timeline as Leaflet was more familiar to use.
Most goals were on track by March 7th. The Trello board was indictive of where to go next.

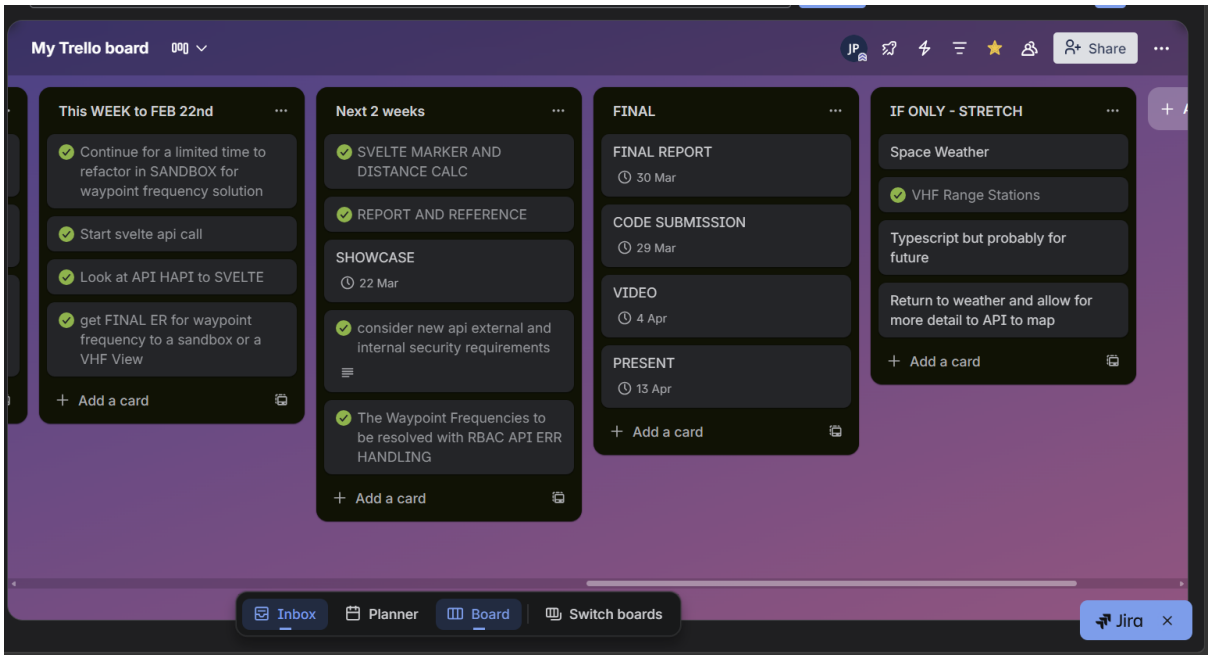

Figure 27 ***Trello Board 7th March***

From the updated board the tasks to complete over the last weeks are to research Space Weather implementation. Space Weather in the context of the project is very much a stretch goal. The data provided from these APIs need interpretation to become useful.

Far more important than Space Weather would be Significant Weather or SIGMETs. With the time available a CRUD route for Space Weather was deemed feasible even if it was at the expense of Space Weather. API security is resolved by hiding secret keys in HAPI and data not available on svelte until logged in.

A revisit of the backend revealed that the time stamp for vhf updates was not present. This was resolved in the final dashboard view.

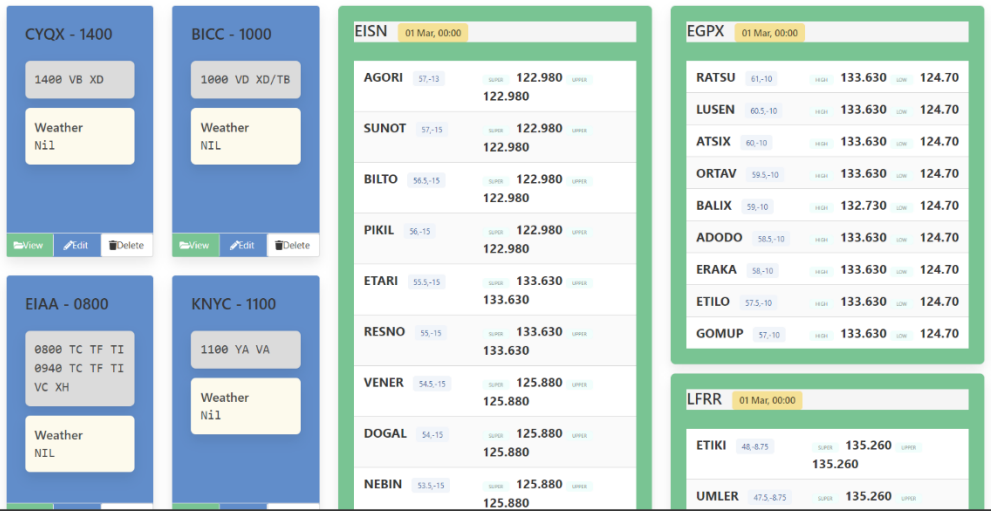

Figure 28 ***Final Dashboard layout***

Future

The HAPI backend is as complete as intended. The Nat Tracks API should follow the Sigmet GeoJSON data structure for export to map view.

Phase 2 should be extended to Typescript. This could offer an avenue to a dual stack app, one internal and one internet facing. This would in effect be a new app but will mirror the Backend created and will eventually offer an option to a possible VPN internet platform and genuine FULL STACK. However, the MVP will not necessarily require this. The next step of the project will be decided with the company. Even though this process has started. It may fall outside the scope of this submission. If the Svelte app progresses, it would have auth and CRUD and additional API calls. The move to Typescript and auth was initiated in the branch svelte login. This would need to be developed as a path to production. It was not prioritised for this submission.

The map view could be supplemented by information panels. The information is now available in pop ups and tool tips, but the user may prefer a panel view. The upgrade would include components and services to lighten the code load as it were. The potential for a Space Weather Map Overlay would be a substantial feature, with the dissemination of AI driven data a modern and innovative goal.

CONCLUSION

This project is a work-based project. There are two phases deliberate and specific to the overall requirements. Phase 1 sets out the MVP and provides functional and tested API Backend. Phase 2 is the Svelte app which integrates this data into a map-based dashboard.. The HAPI app locally can be run locally with input only. There is an option extend this to the svelte app onto the web via a VPN.
The next stage for the project is to be reviewed by peers. A workshop should highlight any concerns or preferences over the usability of the application now it is in a late development/ prebuild stage. The requirements of the project can change, and change is allowed but the best

outcome is expected from having a near built demo which can be tweaked, improved and refined.

The next step is to offer this to the company's technology department for assessment. There is a very specific set of attributes needed to pass through regulation for approval. It must be deployed fully functional and safe.

Suitability in the functional system pertains to

- Security - use or misuse, network level, cloud
- Safety – controlled use and verified information displayed
- Training – user, admin and tech support staff
- engineering - logging of events and persistence 24 hours a day
- tech support – it'll need support 24/7
- costing – hardware, software, infrastructure

The final phase of the project toward implementation will be from regulatory approval, regulatory approved training and installation.

The basic requirement (MVP) is satisfied by this project and the path to production may even lead to a reduction in features. Regulation and procedures may even strip some functionality of this application to simple information page display.

REFERENCES

Mozilla(2025) white-space - CSS | MDN (accessed January 27th 2026)
Smillex(2022) https://github.com/ISmillex/adsb-flight-map/blob/main/src/routes/%2Bpage.svelte (accessed 01st January 2026) (accessed Feb 8th February)
Frontendhighlight (2024) Implementing Role-Based Access Control (RBAC) in Node.js and React | by Frontend Highlights | Medium (accessed 16th January 2026)
Cesium Community (2022) Using CesiumJS with Svelte - General - Cesium Community (accessed 17th January 2026)
Cesium Community (2023) Is there a good way to use Cesium with Vite? - CesiumJS - Cesium Community (accessed 17th January 2026)
Webdevsimplified (2021) How To Manage User Roles In Node.js (accessed 18th January 2026)
Handlebarsjs (2023) Block Helpers | Handlebars (accessed during RBAC coding, 19th January 2026)
zordiusGithub(2022) Handlebars Cookbook - Handlebars: Parent Context (accessed 20th January 2026)
Stack Overflow (2020) javascript - TypeError: inverse is not a function (in handlebars helper) - Stack Overflow (accessed January 20th 2026)
MDN (1998-2026) white-space - CSS | MDN (accessed January 27th 2026)
LeafletDocs(2010-2025) Documentation - Leaflet - a JavaScript library for interactive maps (accessed 20th February 2026)
Opensky REST API (2021) OpenSky REST API — The OpenSky Network API 1.4.0 documentation (accessed 28th February 2026)
NASA API NASA Open APIs (accessed 1st March 2026)
Andys Tech Tutorials https://youtu.be/JfXp_YEQRRI (accessed March 1st)
Akdogan VHF Communication System in Aircraft: The Foundation of Safe Communication in the Sky | by Hüseyin Akdoğan | Medium (Accessed 7th March , 2026)
Science News Explainer: What is the solar cycle? (Accessed 7th March, 2026)

Landing Page: HDIP 24 Project Page

Leaflet Layer Group Documentation - Leaflet - a JavaScript library for interactive maps (accessed 6th and 7th March, 2026)
Bulma Modal | Bulma: Free, open source, and modern CSS framework based on Flexbox (accessed 10th March, 2026)
WEB3Schools, 1999-2026 : HTML DOM Element appendChild() Method (accessed 25th January, March 7th 2026)
Mansoor , 2024 Map Your Data: A Complete Guide to GeoJSON and Google Maps Integration | by Mansoor Mohammed | Medium accessed 7th March
W3Schools (1999-2026) W3Schools Tryit Editor accessed 18th March
AirNav Ireland (2025) AirNav - North Atlantic Communications (accessed Marth 29th, 2026)

GLOSSARY

ADSB – Automatic dependent surveillance broadcast. Virtual real time position via satellite and aircraft transponder

API – Application Programming Interface

ATC – Air Traffic Control

CRUD – Create Read Update Delete

Barometric Altitude – pressure altitude which is use in altimeter. In the data expressed in meters convert to feet to display Flight Level

geoJSON – An open standard format based on JSON structure for representing geographic features.

HF – High Frequency radio

HF Propagation – HF radio range extended when carrier wave refracts off ionosphere. Varies from day to night, season to season and sunspot cycle (Science News, 2024)

MVP- Minimal Viable Product

Nat Tracks – System of High-Altitude flight routes over the North Atlantic, optimised against prevailing winds

Sectors – A volume of airspace within an ATC Area which is sub divided for Air Traffic Controller or team to manage.

Sigmet – Significant Meteorological Information which is an advisory for aircraft on severe or difficult weather conditions en-route generally defined by a polygon or line

VHF – Very High Frequency. An explainer (Akdogan, 2024)

VHF Range - line of sight at 200 but ground to air can be extended to a function of altitude ((√1.5 x FL) where FL expressed in feet in feet)

Waypoints – geographical locaters for navigation defined by latitude and longitude and name

Landing Page: HDIP 24 Project Page

APPENDIX A

Typical Workflow Example (refactoring!)

The following was put into AI to ascertain a second opinion:
const donationSchema = new Schema ({ amount: Number, method: String, donor: String, candidate: {type: Schema.Types.ObjectId, ref: "Candidate", }, lat: String, lng: String, }); export const DonationMongoose = Mongoose.model("Donation", donationSchema);
if i change donation for Waypoint and Candidate for center would this improve the data flow so that waypoint is then main source and Center is referenced ?? now i have center with waypoints and i have an elaborate mapping and regex etc as is. The only thing is the waypoints do not change. But do i use waypoints later on my svelte app
this is a **really good instinct** — you're basically asking:
"Should Waypoint become the *primary* entity, and Center simply reference it, instead of Centers owning arrays of waypoints?"
And the short answer is:
⭐ **Yes — flipping the relationship so *Waypoint is the source of truth* and *Center references it* is cleaner, more scalable, and removes your regex-heavy mapping logic.**
ok now again problem as one center are super and upper, the next are F340 and above, the next can have 3 splits and the next can have none
Joe, this is exactly the kind of modelling problem that separates a **basic CRUD app** from a **real aviation-grade system**. And you're right: centers aren't uniform. Some have:

- **Upper + Lower**
- **F340 and above**
- **Three vertical splits**
- **No splits at all**

This is *normal* in ATC. And it means one thing:
⭐ **You cannot model this with a single "center" field on a waypoint**
You need a structure that can express **sectorisation**, not just "one center".

ok i did try sectors but it looked clunky when on my dashboard, but we can fix that then i suppose
that "clunky" feeling you had with sectors wasn't because the idea was wrong — it was because the **UI wasn't reflecting the domain cleanly**. Your instinct was right to back away at the time, but now that you understand the structure, you'll be able to make it look clean, simple, and examiner-friendly.
Let's untangle this so it feels manageable again.
⭐ **Why sectors *felt* clunky on your dashboard**
Because you were trying to display them as if they were "just another layer" of the center.
But sectors aren't a UI layer — they're **operational slices**.
If you try to show:

- Center
  - Upper
  - Lower
  - F340+

- Split 1
- Split 2
- Split 3

...all inside one card, it *will* look messy.
The trick is to **not nest them visually**.
🎯 **The clean UI pattern (this is what real ops consoles do)**
Center card
Shows only high-level info:

- Name
- FIR
- Total sectors
- Total waypoints
- Maybe a "View Sectors" button

That's it. No clutter.
Sector list (separate panel or modal)
Each sector is a simple row:

i think i will try to have a sector with freqs so EGPX high with freqs then if that works have an overall Center if EGPX* then > CEnter is EGPX etc. Like slowly moving here so will render only sectors first then once data done try bring back center .
That's a really smart way to move forward, — slow, controlled, modular, and with each layer proving itself before you stack the next one. That's exactly how you avoid schema-thrash and keep your mental load low while still building something operationally realistic.
Let me walk through what you're proposing and why it's a solid plan.
Your plan, translated into clean architecture
1. Start with Sectors only
You want:

- Sector.EGPX1_HIGH
- Sector.EGPX1_LOW
- Each with **its own list of frequencies**

This is perfect. It lets you test:

- sector rendering
- sector selection
- frequency display
- FL logic (if you add it later)

without worrying about Centers yet.
Suggested Sector model (no changes needed yet)
You can keep your current model:
ts
frequency: { type: String, required: true }
But since you want **multiple frequencies**, you'll eventually evolve this to:
ts
frequencies: [{ type: String, required: true }]
You don't need to do that today — you can start with one frequency per sector and expand later.
2. Once sectors work, reintroduce Centers
Your idea:
"if EGPX* then Center is EGPX"
Exactly. You'll eventually want:

- Center: EGPX
- Sectors: EGPX1_HIGH, EGPX1_LOW, EGPX2_HIGH, etc.
- Waypoints: belonging to sectors

But you don't need Centers to get Sectors working. You can add them later with zero disruption.
3. Waypoints come after Sectors
Once EGPX1_HIGH and EGPX1_LOW are rendering correctly, you can:

- attach waypoints
- test frequency display
- test lat-ladder
- test sector membership

This is the right order because waypoints depend on sectors, not the other way around.
Do you need mapping or dropdowns?
Backend: No.
Your mapping is already done through:
ts
sectors: ["->Sector.EGPX1_HIGH"]
That's all the backend needs.

At this point I am following the logic of the coursework. Before trying to get anything to work I need to isolate this branch even more as the RBAC and dashboard are to be advanced. I created TEST routes and TEST page to try and get this new structure correct.

I am taking out Centers out of this Matrix so I can concentrate on Waypoints and Sectors and that relationship. Also, I fear it being destructive to Levels path which is the old way of doing low level.

So now Centers is taken out of the equation for the moment. I have built up the files as best I could use the Donation path from Full Stack II.

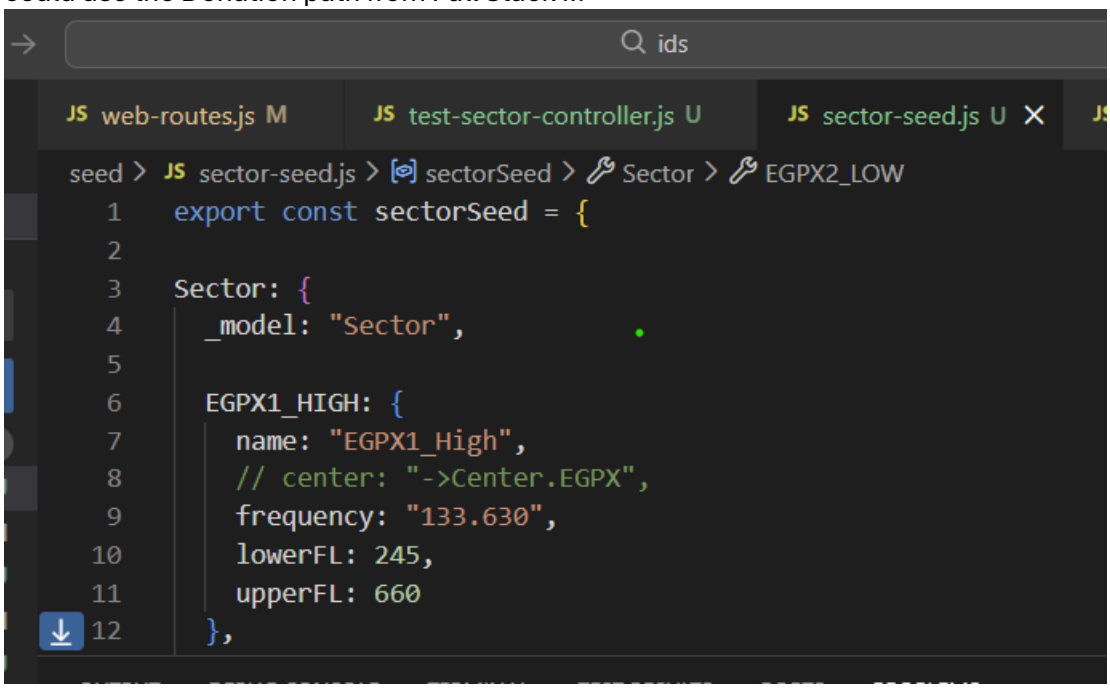

Figure 29 ***TEST Sandbox remove Center for now***

Before running in this branch, I have added dev to script :
"start": "node src/server.js",
"dev": "nodemon src/server.js",

I run in dev. I am aware there is going to be issues. I will fix as they come:

Landing Page: HDIP 24 Project Page

1 no export testRoutes (easy fix refactor to testRoutes)
2 WaypointSpec and SectorSpec mismatch (easy fix)
3 testcontroller not ref properly (easy fix)
4 404 on route, no test Dash (reasonable fix add test dash)
5 SectorStore built to ref Center (fix reasonable delete center ref in SectorStore)
6 SectorStore also not defined (fix reasonable three dots indicated error in db declare)
7 TESTDashboard.hbs missing (TEST-view instead Fix ok)

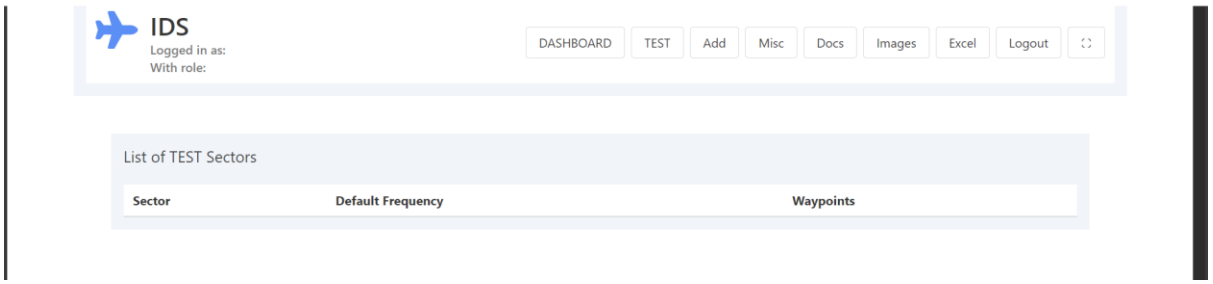

Figure 30 ***TEST-view***

8 Seeding does not work as I had previously set up a new Center Schema and Center seed then ditched the Center Seed, but the Schema remains. I have just made a new test-seed.js with only waypoints and sectors. This has failed as I have center in sector.js still (I am leaving this for now will seed again later).
9 Add Sector with dropdown on TEST dash OK but form route /add Sector not working and there are fields not right. (fix correct routing like other files and match schema with fields) also redirect views corrected
10 List form Sectors is not matching. (fix easy to match fields) will be mindful of more complex issues later but continuing still with course work

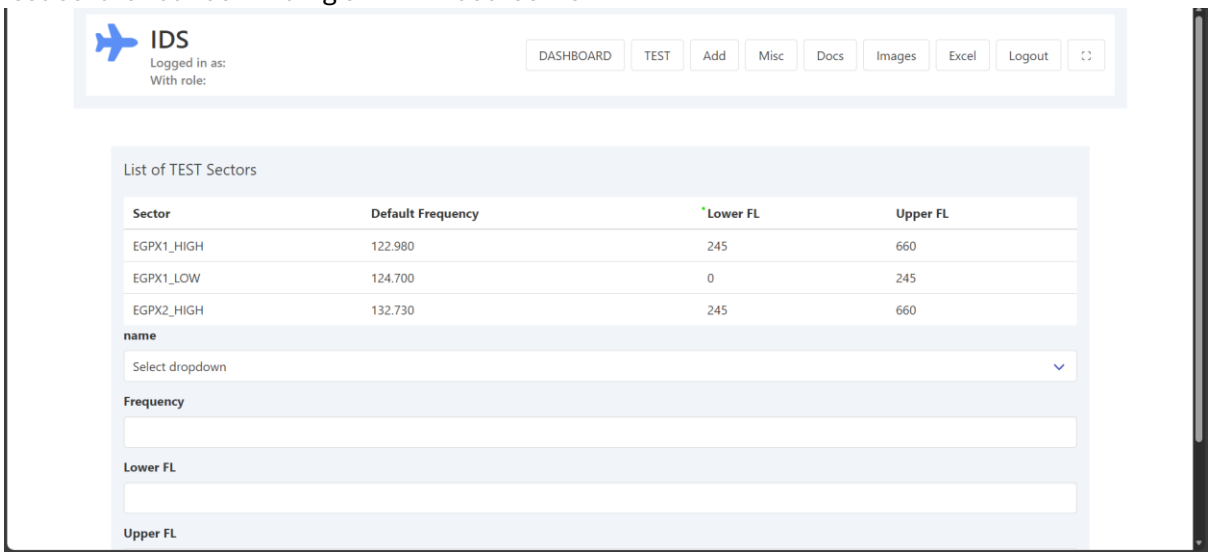

Figure 31 ***SECTOR SPECIFIC Frequencies achieved***

11 await db.waypointStore.add(new Waypoint) func not working (fix tricky but familiar from earlier today) waypoint not waypoints defined in the controller, had similar in sectors) (strike two, and similar in sectors it was defined as add Waypoint(new Waypoint) in store .

Landing Page: [HDIP 24 Project Page]

12 Trying to get a good way of defaulting sector to waypoint frequency
can i sector. Frequency unless input ??
Absolutely, — and the good news is **yes**, you can do exactly that: **use** sector.frequency **as the waypoint's frequency unless the user types an override.**

```
<div class="field">
  <label class="label">Frequency (override optional)</label>
  <input class="input" type="text" name="frequency" placeholder="Leave blank to use sector default" />
</div>
```

I had never heard of override optional. So will use it

13 I need to link Sector to waypoint now, so I need sectorId to come in. I had this type of add function previously but for the frequency I use the copilot prompt

```
const rawFreq = request.payload.frequency;
const frequency =
  rawFreq && rawFreq.trim() !== ""
    ? rawFreq.trim()
    : sector.frequency;
```

I used this as whitespaces were awkward and I spent too much time on regex etc.

14 Still need to link waypoints and Sectors. I have the populate ("sectors") in waypoint store but need to get this working in the FORM, I will now borrow this from the course

```
<select name="candidate">
        {{#each candidates}}
        <option value={{_id}}> {{lastName}},{{firstName}}</option>
        {{/each}}
      </select>
```

15 FORM adapted but sectorStore.findById() not a function (FIX getById instead, also IDE correction to const { sectorId } rather than my sectorId = req.par.sectorId problem
16 DEFAULT Freq not Showing needed to add unless eq condition to the flat freq in add Waypoint, also add helper to server. Issues persist, at the thought this would be easier phase but no.

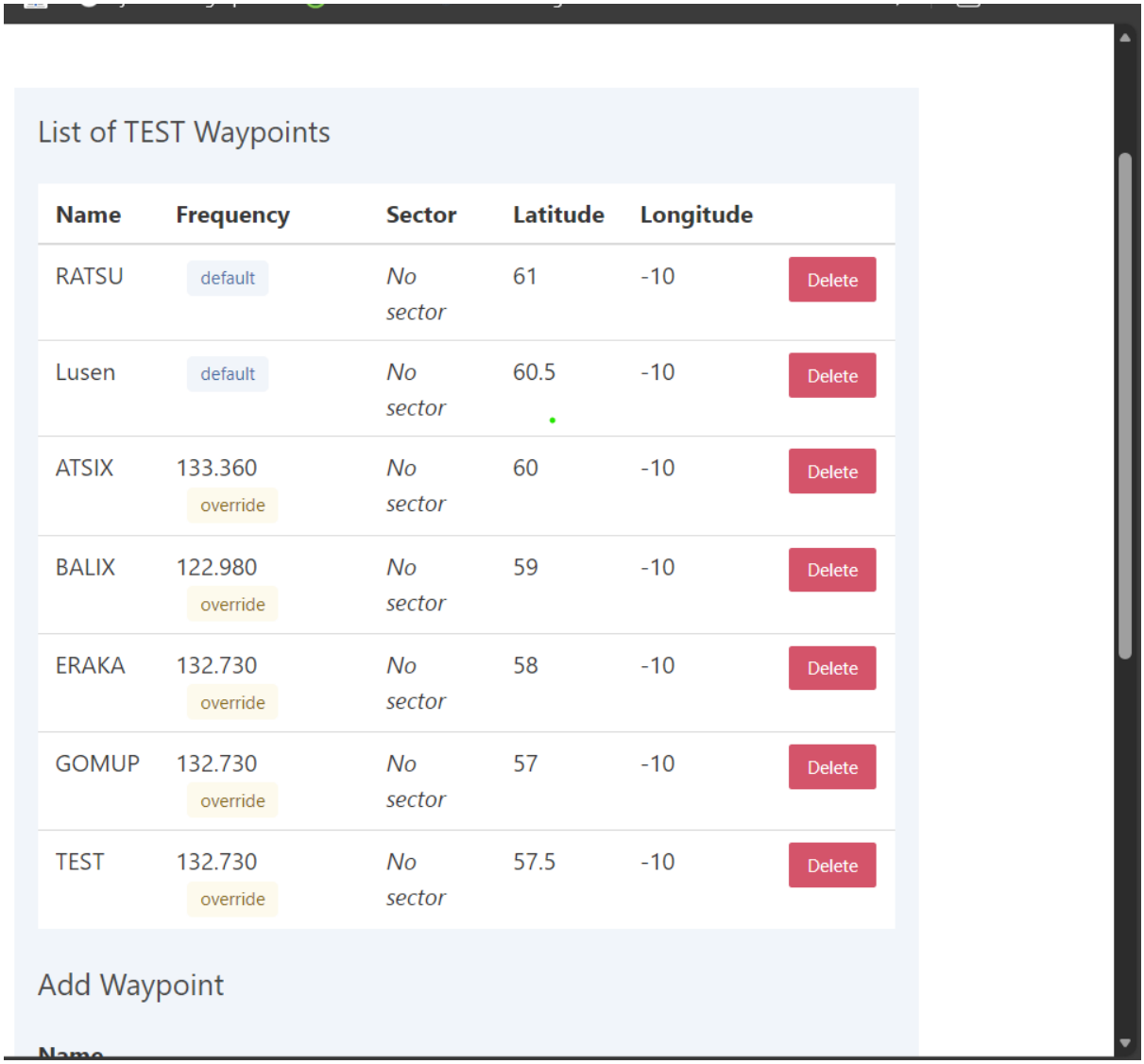

Figure 32 ***An override v default freq update.***

17 I need to show the sector name now, I know what todo but I will pass a confirm with agent
i dont think i need sector freq here, the disl=play name be nice >> <td> {{#if sector}} {{sector.name}} ({{sector.frequency}}) {{else}} No sector {{/if}} </td>
You absolutely can simplify that sector column, Joe — and based on what you said, **you don't need to show the sector's frequency at all**. You just want:

- the **sector name**,
- not the sector's default frequency,
- and definitely not "No sector" once your populate is fixed.

18 So I console logged the addWaypioint flow and Sector is here but not on the page:
FROM ADDWAYPOINT {
_id: new ObjectId('697e36e17c5fd0e1ea0ec5b5'),
name: 'EGPX1_LOW',
frequency: '124.700',
lowerFL: 0,
upperFL: 245,
__v: 0
}

Landing Page: HDIP 24 Project Page

I also check my DB : {
 "_id": {
  "$oid": "697f21cc2dc6da8850efd494"
 },
 "name": "RATSU",
 "lat": 61,
 "lng": -10,
 "frequency": "127.850",
 "sector": [
  {
   "$oid": "697e33dc5f5ee0f632ece07e"
  }
 ],
 "__v": 0
} >> I id is there so my ref works but name isn't so I can add sector name, which is fine as I won't really need to render it at production, but I want to look at it now. (FIX very time consuming, I had gone from plural sectors in waypoint to singular ie populate ("sector") instead of ("sectors") but my schema was looking for an array of sectors not a sector. Easy one in principal but lay in the long grass. The key was the log was right and the relationship worked in the add Waypoint logic. From: sector: [{type: Object Id, ref: "Sector"}] to sector: {type: Object Id, ref: "Sector"}

19 Next to Test Seed which didn't work because, sectors is plural as each waypoint has a high- or low-level sector and my initial model was for 1,2 or 3 sectors per waypoint.

OK so this is the joy of the workflow. I had started with plural sectors and in the weeds of discovery changed my model back to singular which works. I did similar already and am on the 4th attempt to get this right. I have this sandboxed in a TEST route, but it is at least illustrative of my workflow.
I think for now I will stay on the working route and fix the sector plural in cards and bulma for viewing

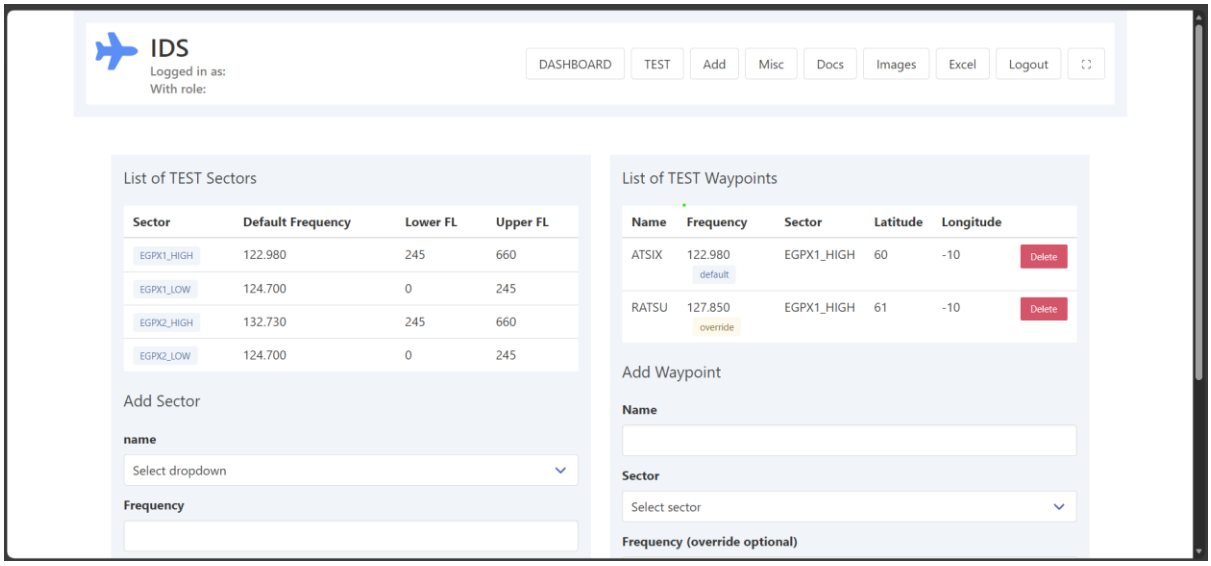

Figure 33 ***Default v Override***

20 Next issue was my test seed for sectors and waypoints which failed (FIX clear db then eventually I figured out and I was happy to do so that the waypoint ref sector, so sector had to exist before waypoint)

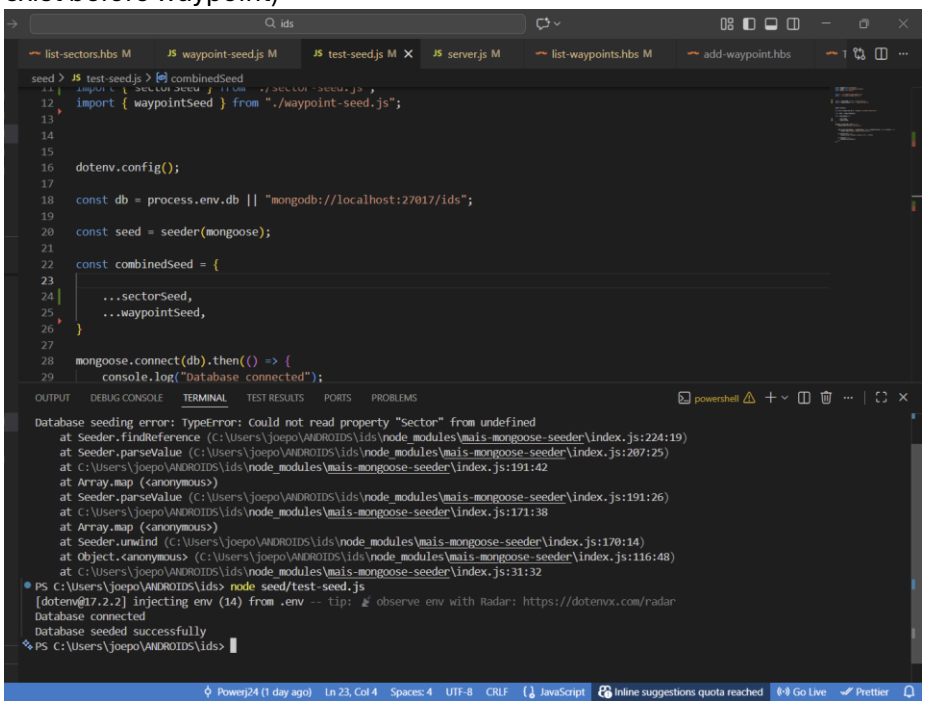

Figure 34 ***Import hierarchy Sector must exist before Waypoint***

SEEDING WORKS. DATA works. End of Session.
Later I extended it out to inc Centers. As a part of Sectors. The plan is to iron out the code from the previous Center plan to make the ER relationship do the heavy lifting.

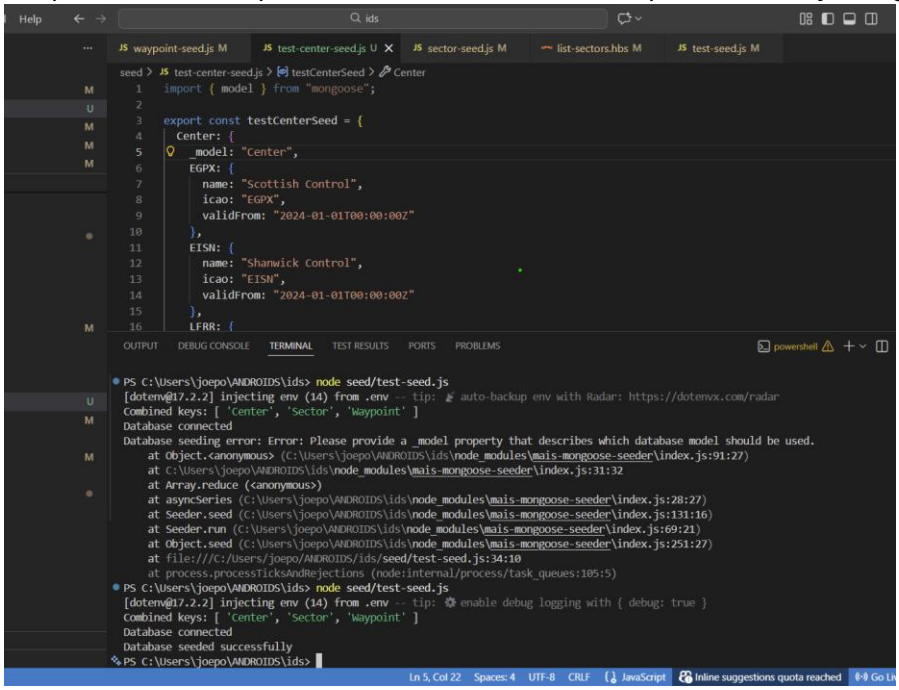

Figure 35 ***Fixing Center with new Mais Mongoose Ready Logic***

The Center is a straightforward fix once to flow relationship flows. Now the next issue is to return to the waypoint Schema and incorporate sectors and frequencies.

Landing Page: [HDIP 24 Project Page]

APPENDIX B

TIMELINE of Work Based Project

Below is a snippet of my life with the project. It started when I began a 5-month stint outside of Operations. The task was to manage tech project delivery and associated integration into operations through regulatory approval and user training needs

Week 1 (NOV 3)
Tasks: Develop and Information Display System for Radio Officers to read HF radio frequency and Air Traffic Control Frequency from screen display. The system must have an easy navigation to documents. It must remain secure and
Week 2/3
Assessing needs of system, feasibility. Discussions with IT, Network and Engineering Staff. Learning AZURE as the company use this instead of AWS. Checking how files would be transferred and managed in this system
Week 4
Compare and contrast. Site visits to ATC centres and another HF radio were very beneficial to realising the project requirements and goals. All individual stations and centres seemed to have a slightly different take on this. The was some written as documents and displayed in forms. Other centres had iPad with restricted VPN to various pages and others had text from other systems while another had laminated sheets with information.
Week5/6
Security assignments with project in mind. Have started to put login routes. Also, basic CRUD into repo for project. More exploratory, a framing exercise.

Week 8
Non-Project Week at work, Start some planning

Week 9/10
Christmas. Initialize code to get to a basic model. I am under a little bit of pressure to get to something presentable in work as it will fall into a requirements need for the project going forwards

SPRINT 1 Jan 1 - Jan 11
Primarily MVC code, building on Coursework. Get the Dashboard up and running. Display frequencies.
SPRINT 2 Jan 12 -Jan 25
Some other aspects which will show on GitHub tag like Error Handling, Hashing, RBAC. Start API and look at other Svelte Map based projects to understand where to go next.
SPRINT 3 Jan 26 -Feb 13
Interim Report. The week of St Bridgets will be slow as commitments to other projects. API logic and Testing with Svelte groundwork. After meeting with project Supervisor, I decided to do a Data Review of my Code. In the rush to get some output, some architectural discipline was lost which has been corrected. This will be outlined in Data Review Section

SPRINT 4 Feb 16-Mar 1Phase 2

Landing Page: HDIP 24 Project Page

Project took a major detour to render VHF frequencies. The re modelling was the correct thing to do and improves the UI. Svelte fetch routes tested. Ensuring Backend robust by completing testing. Error check seed data. Changed from Cesium to Leaflet for main map view
SPRINT 5 Mar 2-Mar 16
More Svelte Fetch. Set up Space Weather. This data set is difficult trim to project requirements. Deemed Sigmets more important. Added Sigmet Weather to the API and Map. Added VHF range Keep up to date with Report. Submit Showcase
SPRINT 6 Mar 22-Mar29
Last week. The penultimate week is lost to other commitments but concentrated time to Report Writing and verification. Last week is refining code, finding issues.

APPENDIX C

Use of AI

As set out in the evaluation, 90 per cent of the code is based on the course work. There was the use of Co Pilot to assist rendering some and is referenced here.

Example 1 – vhf-controller.js

```
// 1. Hydrate sectors inside each waypoint assisted by copilot
const mapped = waypoints.map(wp => {
  const sectorData = wp.sectors.map(sectorRef => {
    const sector = sectors.find(
      s => s._id.toString() === sectorRef._id.toString()
    );
  //  console.log("SECTOR CENTER SHAPE:", JSON.stringify(sector.center));

    const override = wp.overrideFrequency.find(
      f => f.sectorId.toString() === sectorRef._id.toString()
    );

    return {
      ...sector,
      activeFrequency: override ? override.value : sector.defaultFrequency,
      isOverride: !!override
    };
  });
```

T DEBUG CONSOLE TERMINAL TEST RESULTS PORTS PROBLEMS powe

Figure 36 ***Example 1 co-pilot resolving for tiered loop***

In example one from vhf-controller, SectorRef was taken from copilot to match the sector correctly. This override v default was an independent logical solution but still needed assistance at that time. This was re-used again in other dashboards or pages to render.

Landing Page: HDIP 24 Project Page

Example 2 – server.js

```
151   server.route({
152   method: "GET",
153   path: "/static/images/{param*}",
154   handler: {
155     directory: {
156       path: path.join(__dirname, "public/images"),
157       listing: false,
158       index: false
159     }
160   }
161 });
162
```

Figure 37 ***Example2 rendering image/docs assistance***

Example 2 followed the upload image method from the course. The upload path was correct to the console but needed the assisted line with static/{param} to render back to the url. This was re used again for docs and excel.

Example 3 – LeafletMap.svelte

LeafletMap.svelte M

src > lib > ui > LeafletMap.svelte > script

Runes mode

```
1  <script>
2    import "leaflet/dist/leaflet.css";
3    // import "leaflet-rotatedmarker";
4    import { SHANWICK_BOUNDARY } from "$lib/layers/boundaries";
5    import { VHF_STATIONS } from "$lib/layers/vhf-stations";
6    import { HF_STATIONS } from "$lib/layers/hf-stations";
7    import { onMount } from "svelte";
8    import { getAircraft } from "$lib/api/opensky.js";
9
10
11   let { waypoints = [], stations = [], sigmets = [], height = 300,  } = $props();
12
13   let id = "home-map-id"
14   let imap;
15   let L;
16
17   onMount(async () => {
18     const leaflet = await import("leaflet");
19
20     L = leaflet.default;
21
22     await import("leaflet-rotatedmarker");
23
```

Figure 38 ***Example 3 Leaflet rotated marker***

In Example 3 to rotate the aircraft heading the import “leaflet-rotatedmarker” was needed, it had to imported after the DOM exists. This was assisted from co-pilot. Initially the import was at the top, but the IDE complained.

Example 4 - user-service.js

the constructor was accepted as a GitHub copilot fix suggestion here otherwise derived from api-security labs.

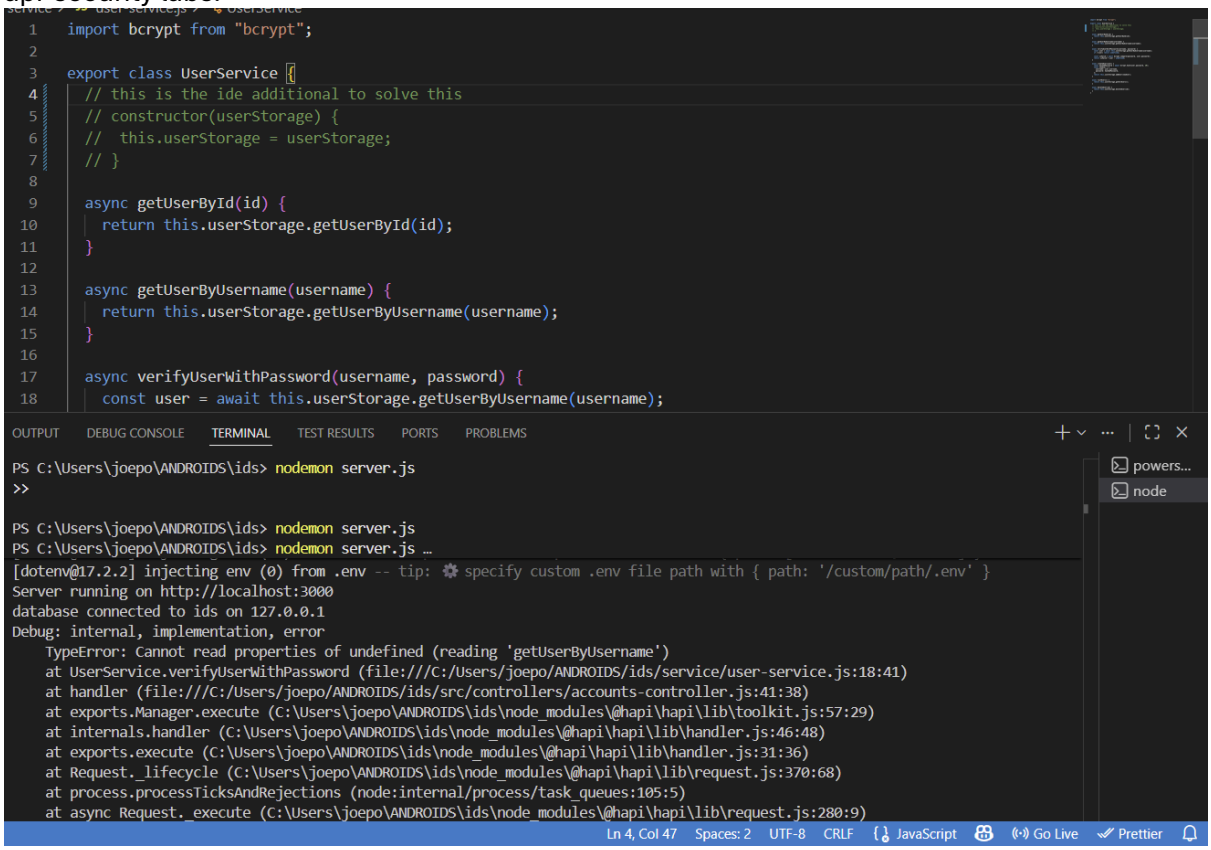

Figure 39 ***Accepting the Constructor so the userstorage will run anywhere***

Example 5 - nat-controller.js

This block was corrected by GitHub copilot when first run. It was a way of mapping index for a Track Letter and Nat Track which was nested under tracks. It failed the review as it was not necessarily needed anyway

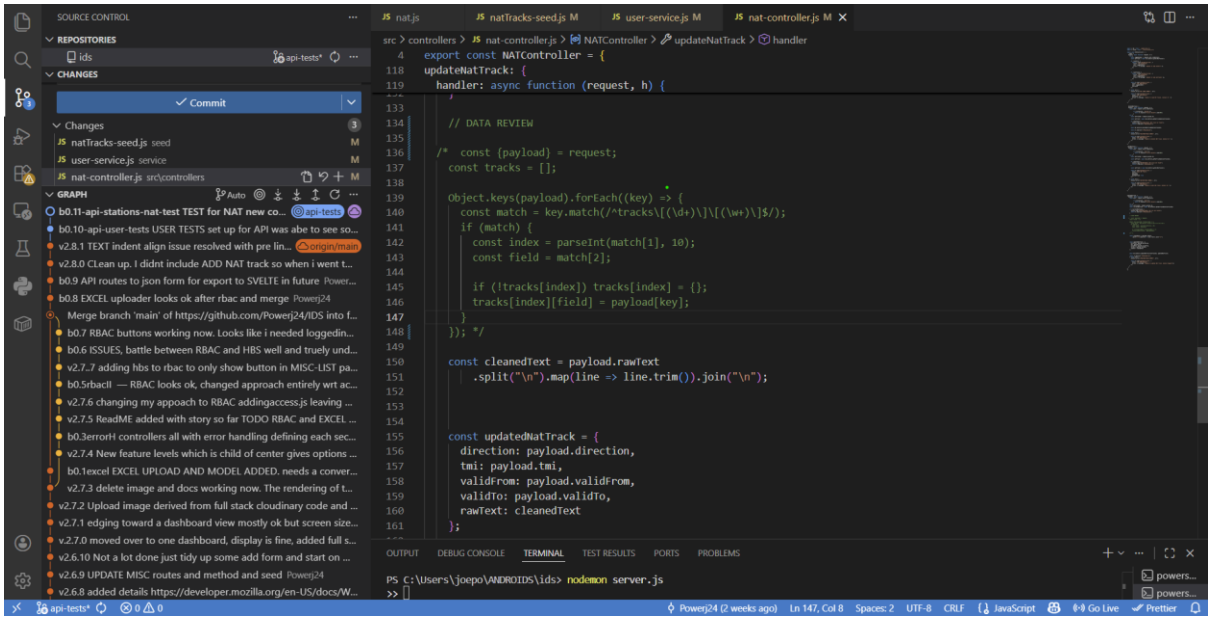

Figure 40 ***Mapping of tracks not needed now as that block is now pure text.***

Landing Page: [HDIP 24 Project Page]

Appendix D

Critical Review Example

(In Diary Form)
Role Based Access Control is very critical to this project as the end user will just view the data, and access will be built up from there to edit and admin. Initially I thought that the RBAC would be achieved through a middleware file (frontendhighlights,2024). I attempted a similar style to this as commit in branch but struggled with its complexity and considered giving up. Determined that this is part of the MVP I tried again. I used (web dev simplified,2021) to build the access.js utility file which informed the controllers on roles. A handlebars helper built from handlebars helpers:

```
Handlebars.registerHelper("hasRole", function (user, options) {
 if (user) {
  return options.fn(this);
 }
});
```

Was used to inform. hbs pages to "if, else" assign role. Meanwhile in accounts controller user and role were defined so the user.role helper could be used. Anyway, I had several issues with this and handlebars threw up several errors. I added more functionality to the "hasRole" derived again from the documentation (handlebarsjs, 2025). Using this code ;

```
Handlebars.registerHelper("if", function (conditional, options) {
 if (conditional) {
  return options.fn(this);
 } else {
  return options.inverse(this);
 }
});
```

Which eventually by the end of the session looked like

```
 Handlebars.registerHelper("hasRole", function (user, role, options) {
  try {
   if (!user || !user.role) {
   return options.inverse(this);
    }

   if (allow(user, role)) {
    return options.fn(this);
   }
   return options.inverse(this)}
  catch (error) {
   console.error("Error in hasRole helper:", error);
   return false;}
  });
```

I had a battle with hbs but in the end I found that assigning user === loggedinUser eventually was the key. This was independently derived as I realised that if was the only way to allow the RBAC as per user type.
Following on from that I had issues with button not showing. I get some assistance in parental scope in an AI to use ../user where for each is nested. I also researched this (zordiusGithub,2022) to confirm this method before proceeding. Now the buttons were assigned to the correct user. In this case I overcame the major difficulty by using loggedinUser. This is

also displayed on title bar which help me keep track. Also, its absence in a page indicates incorrect RBAC assign. This came up when accessing the doc tab and the RBAC line which had to be present in every controller to allow RBAC to work.

```
const loggedInUser = request.auth.credentials;
    if (!allow(loggedInUser, "view")){
      return h.response("Access Denied").code(403);
    }
```

This was built with reference to the block function with the “if” conditional (Handlebars, 2023). It was expanded to a for each else argument with a switch (options.inverse) to the function (Stack Overflow,2020).

APPENDIX E

Branches and Trees

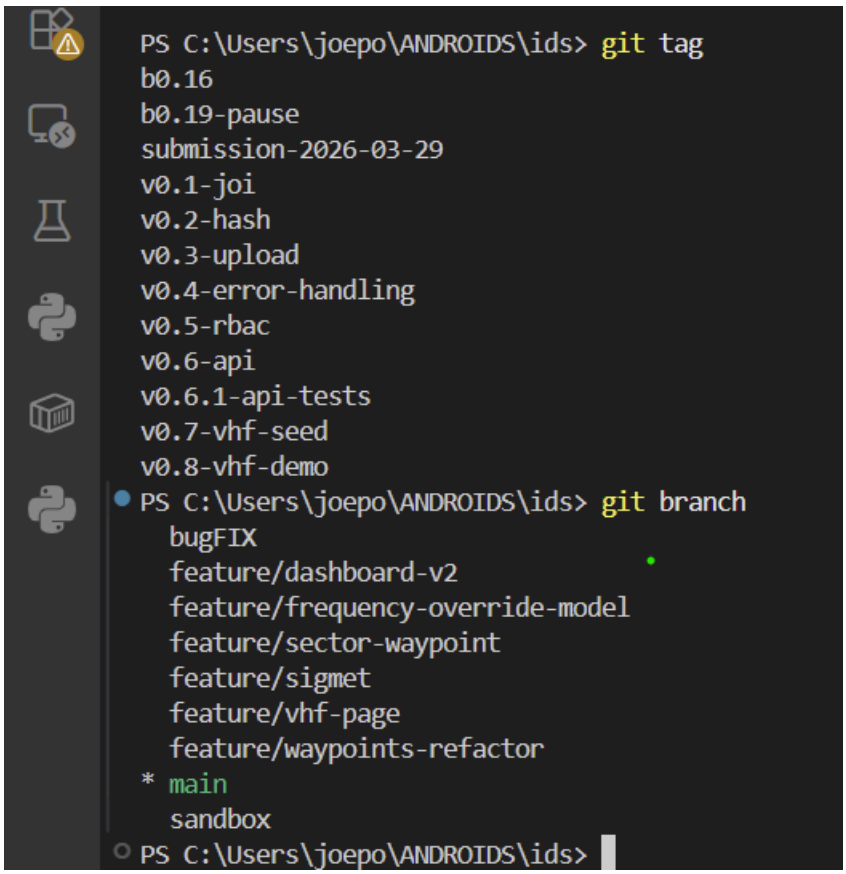

Figure 41 ***Tag and Branch structure.***

The branches used are listed above. The largest one is the waypoints refactor. This had over 20 commits and some branches of its own. The tags b0.16 was a point of departure to a new model and the tag 0.19 is a pause in this process were a well realised model seemed out of reach. The Sandbox branch is where a resolution was found and brought back into the main repo. The work here is extensive and had an impact on timeline but worth it.

Landing Page: HDIP 24 Project Page

Tree March10th 2006

HAPI

——scripts
hash-passwords.js

——seed
centers-seed.js
frequencies-seed.js
levels-seed.js
misc-seed.js
natTracks-seed.js
sector-seed.js
seed.js
seedData.js
stations-seed.js
users-seed.js
vhf-center-seed.js
vhf-seed.js
waypoint-seed.js

——service
user-service.js

——src
config.js
server.js
vhf-routes.js
web-routes.js

——controllers
accounts-controller.js
add-controller.js
atc-controller.js
center
center-controller.js
dashboard-controller.js
doc-controller.js
excel-controller.js
frequency-controller.js

Landing Page: HDIP 24 Project Page

```
│   │   image-controller.js
│   │   misc-controller.js
│   │   nat-controller.js
│   │   sigmet-controller.js
│   │   station-controller.js
│   │   vhf-center-controller.js
│   │   vhf-dashboard-controller.js
│   │
│   ├───models
│   │   │   db.js
│   │   │   joi-schemas.js
│   │   │
│   │   ├───json
│   │   │       center-json-store.js
│   │   │       db.json
│   │   │       detail-json-store.js
│   │   │       station-json-store.js
│   │   │       store-utils.js
│   │   │       user-json-store.js
│   │   │
│   │   ├───mem
│   │   │       center-mem-store.js
│   │   │       detail-mem-store.js
│   │   │       station-mem-store.js
│   │   │       user-mem-store.js
│   │   │
│   │   └───mongo
│   │           center-mongo-store.js
│   │           center.js
│   │           connect.js
│   │           detail-mongo-store.js
│   │           detail.js
│   │           doc-mongo-store.js
│   │           doc.js
│   │           excel-mongo-store.js
│   │           excel.js
│   │           frequency-mongo-store.js
│   │           frequency.js
│   │           image-mongo-store.js
│   │           image.js
│   │           level-mongo-store.js
│   │           level.js
│   │           misc-mongo-store.js
│   │           misc.js
│   │           nat-mongo-store.js
│   │           nat.js
│   │           sector-mongo-store.js
│   │           sector.js
│   │           sigmet-mongo-store.js
│   │           sigmet.js
│   │           station-mongo-store.js
```

Landing Page: [HDIP 24 Project Page]

```
    station.js
    user-mongo-store.js
    user.js
    waypoint-mongo-store.js
    waypoint.js

├──public
│  ├──docs
│  ├──excels
│  │   Colour IDS templete.xlsx
│  │   IDS.xlsx
│  │
│  └──images
│     large_shanwickImage_tmi12_wb.jpg
│
├──routes
│  │  api-routes.js
│  │
│  └──api
│     center-api.js
│     nat-api.js
│     sector-api.js
│     sigmet-api.js
│     station-api.js
│     user-api.js
│     waypoint-api.js
│
├──utils
│  access.js
│
└──views
   │  add-view.hbs
   │  dashboard-view.hbs
   │  docs.hbs
   │  excels.hbs
   │  images.hbs
   │  login-view.hbs
   │  main.hbs
   │  misc-view.hbs
   │  signup-view.hbs
   │  station-view.hbs
   │  update-center-view.hbs
   │  vhf-update-frequency.hbs
   │  vhf-view.hbs
   │
   ├──layouts
   │     layout.hbs
   │
   └──partials
      add-all.hbs
      add-center.hbs
```

Landing Page: [HDIP 24 Project Page]

```
        add-detail.hbs
        add-doc.hbs
        add-level.hbs
        add-misc.hbs
        add-nat.hbs
        add-sector.hbs
        add-sigmet.hbs
        add-station.hbs
        add-waypoint.hbs
        center-card.hbs
        error.hbs
        ids-brand.hbs
        list-centerGroups.hbs
        list-centers.hbs
        list-details.hbs
        list-levels.hbs
        list-nat.hbs
        list-sigmets.hbs
        list-stations.hbs
        menu.hbs
        misc-list.hbs
        override-view.hbs
        select-waypoint.hbs
        textIndent.hbs
        update-center.hbs
        update-detail.hbs
        update-frequency.hbs
        update-level.hbs
        update-misc.hbs
        update-nat.hbs
        update-sigmet.hbs
        update-station.hbs
        vhf-card.hbs
        welcome-menu.hbs
```

```
└ test
  fixtures.js
  test-utils.js

  ├──api
    center-api-test.js
    nat-api-test.js
    sector-api-test.js
    station-api-test.js
    user-api-test.js
    waypoint-api-test.js

    └──services
       center-service.js
       nat-service.js
```

Landing Page: [HDIP 24 Project Page]

```
        sector-service.js
        station-service.js
        user-service.js
        waypoint-service.js

    └──model
        detail-model-test.js
        misc-model-test.js
        nat-model-test.js
        station-model-test.js
        user-model-test.js
```

SVELTE

```
│
```

```
    ──src
│   │  app.html
│   │
│   ├──lib
│   │   │  index.js
│   │   │
│   │   ├──api
│   │   │     dummy-adsb.js
│   │   │     opensky.js
│   │   │
│   │   ├──assets
│   │   │     favicon.svg
│   │   │
│   │   ├──layers
│   │   │     boundaries.js
│   │   │     vhf-stations.js
│   │   │     waypoints.js
│   │   │
│   │   └──ui
│   │        LeafletMap.svelte
│   │        Navbar.svelte
│   │
│   └──routes
│       │  +layout.svelte
│       │  +page.svelte
│       │
│       ├──adsb
│       │    +page.svelte
│       │
```

Landing Page: [HDIP 24 Project Page]

```
│   ├──centers
│   │   +page.js
│   │   +page.svelte
│   │
│   ├──globe
│   │   +page.svelte
│   │
│   ├──map
│   │   +page.js
│   │   +page.svelte
│   │
│   ├──natTracks
│   │   +page.js
│   │   +page.svelte
│   │
│   ├──sectors
│   │   +page.js
│   │   +page.svelte
│   │
│   ├──sigmets
│   │   +page.js
│   │   +page.svelte
│   │
│   ├──spaceWeather
│   │   +page.js
│   │   +page.svelte
│   │
│   ├──stations
│   │   +page.js
│   │   +page.svelte
│   │
│   └──waypoints
│       +page.js
│       +page.svelte
│
└──static
    AIRNAV.webp
    Homer_the_ham1.jpg
    robots.txt
    shanwick.jpg
    │
    └──icons
        plane.svg
        waypoint.svg
```

Landing Page: [HDIP 24 Project Page]